MESSIFY- AN AI-POWERED SMART MESS MANAGEMENT SYSTEM

A THESIS
Submitted by

Abdur Rafey **Saud Asad**
Shruti Dhengle **Manthi Pipale**
Kshitij Meshram **Ajitesh Gupta**

for the award of the degree
of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
Dr. Trupti Meshram



COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING AND TECHNOLOGY
G H Raisoni University, Amravati

2026-27

DECLARATION

We declare that

a. The work contained in the Report is original and has been done by us under the general supervision of our supervisor.
b. We have followed the guidelines provided by the University in writing the Report.
c. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the University.
d. Whenever we have used materials (data, theoretical analysis and text) from other sources, we have given due credit to them by citing them in the text of the Report and giving their details in the references.
e. The Report has been subjected to plagiarism check using professional software and found to be within the limits specified by the University.
f. The work has not been submitted to any other Institute for any degree or diploma.

Name of Students	**Enrollment No.**
Abdur Rafey	GHRUA23011060488
Saud Asad	GHRUA23011060964
Shruti Dhengle	GHRUA23011060619
Manthani Pipale	GHRUA23011060548
Kshitij Meshram	GHRUA23011060218
Ajitesh Gupta	GHRUA23011060

CERTIFICATE

This is to certify that the report entitled

FARMSHARE: A WEB-BASED PLATFORM FOR ONLINE FARM EQUIPMENT RENTAL AND BOOKING MANAGEMENT WITH MULTILINGUAL AI-ASSISTED SUPPORT

is a bonafide work and it is submitted to

G H Raisoni University, Amravati

by

ABDUR RAFEY

SAUD ASAD

SHRUTI DHENGLE

MANTHANI PIPALE

KSHITIJ MESHRAM

AJITESH GUPTA

For the partial fulfillment of the requirement in the degree of Bachelor of Technology in Computer Science & Engineering, during the academic year 2026-27.

Dr. Trupti Meshram
Project Guide

Dr. Suresh Salankar
Dean, Department of CSE

PROCEEDINGS OF EXTERNAL VIVA-VOCE EXAMINATION

The viva-voce examination of Abdur Rafey, Saud Asad, Shruti Dhengle, Manthani Pipale, Kshitij Meshram, Ajitesh Gupta on this B.Tech dissertation titled “Messify- An AI Powered Mess Management System” was conducted in the School of Engineering and Technology in the Department of Computer Science and Engineering at G H Raisoni University, Amravati on 2026.

During the viva-voce examination the following examiners were present.

1. Guide: Dr. Trupti Meshram

2. External Examiner:

Based on the presentation given by Abdur Rafey, Saud Asad, Shruti Dhengle, Manthani Pipale, Kshitij Meshram, Ajitesh, both the examiners approve the dissertation for the partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering.

Signature:

Project Guide: _____

External Examiner: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Trupti Meshram, Department of Computer Science & Engineering, G H Raisoni University, Amravati, for her valuable guidance, continuous encouragement, constructive suggestions and support throughout the development of MESSIFY and preparation of this thesis.

We are grateful to the Department of Computer Science & Engineering and the University for providing the academic environment, facilities and resources required to design, implement, test and document the project. The discussions and feedback received during different stages of the project helped us improve both the technical implementation and the quality of the documentation.

We also thank our classmates, friends and all individuals who provided feedback while reviewing the user flows, mess-management requirements and overall usability of the system. Finally, we express our deepest gratitude to our parents and family members for their continuous support, patience and motivation.

Name of Students	Enrolment No.
Abdur Rafey	GHRUA23011060214
Saud Asad	GHRUA23011060964
Manthani Pipale	GHRUA23011060633
Shruti Dhengale	GHRUA23011060202
Kshitij Meshram	GHRUA23011060956
Ajitesh Gupta	GHRUA23011061045

CONTENTS

Chapter	Illustration	Page
	List of Tables	**i**
	List of Figures	**ii**
	List of Photographs (Screenshots)	**iii**
	List of Symbols and Abbreviations	**iv**
	Abstract	**v**
1.	**INTRODUCTION**	1
1.1	Problem Statement, Objectives and scope	2
1.2	Background and Motivation	4
2.	**LITERATURE SURVEY**	6
2.1	Comparative Analysis	8
2.2	Review of Related Work	10
2.3	Research Gap	12
3.	**METHODOLOGY**	13
3.1	Requirements and System Design	15
3.2	System Architecture	17
3.3	User Workflow	19
3.4	Database Design	21
3.5	Function Modules	24
3.6	Student	25
3.7	Mess Owner	27

3.8 Delivery and Anministration 29

3.9 AI Assistance and Recommendation 30

3.10 Security and Consideration 32

3.11 Technology Stack 34

4. STREANGTH AND LIMITATION 36

4.1 Strength of the project 37

4.2 Limitation of the Project 39

5. RESULT AND DISCUSSION 41

5.1 Test Result 43

5.2 Functional Outcome Analysis 45

5.3 Screenshots Insertion Pages 47

6. CONCLUSION 54

6.1 Future Scope 56

References 58

Research Paper Publication

Certificate of Research Publication

LIST OF TABLES

Table	Title	Page No
2.1	Comparison table of Different App	8
3.1	Functional Requirement Table	15
3.2	Overview of Database Collection	22
3.3	Technology Stack and its Role in Messify	34
5.1	Observed Functional Outcomes	41
5.2	Test Results and System Evaluation	43

LIST OF FIGURES

Figure	Title	Page No.
3.1	MESSIFY System Architecture	17
3.2	Primary Student-to-Delivery Workflow	19
3.3	Firestore Logical Data Model Diagram	21
3.4	Functional Module Diagram	24
3.5	Current AI Assistance and Personalization Flow	30

LIST OF PHOTOGRAPHS

Photograph	Title	Page No.
5.1	Mess Portal and Landing Page	47
5.2	Messify Home Page and User Login Interface	47
5.3	Mess Owner Dashboard	48
5.4	Mess Owner Ledger and Earning Management	48
5.5	Route Assignment Interface for Mess Management	49
5.6	Student Dashboard	49
5.7	Meal Menu and Custom Dietary Request Interface	50
5.8	Student Billing and Payment Leadger	50
5.9	Online Payment Interface	51
5.10	Student Profile and Payment Leadger	51
5.11	Delivery Partner Profile Section	52
5.12	Delivery Partner Delivery History	52
5.13	Delivery Partner Tips and Payment History	53

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol	Description
AI	Artificial Intelligence
CSS	Cascading Style Sheets
DOI	Digital Object Identifier
ER	Entity Relationship
FK	Foreign Key
GHRUA	G H Raisoni University, Amravati
GPS	Global Positioning System
HTML5	Hyper Text Markup Language, version 5
ID	Identifier / Identity
API	Application Programming Interface
JSX	JavaScript XML
MERN	MongoDB, Express.js, React, Node.js
OTP	One-Time Password
UPI	Unified Payments Interface
URL	Uniform Resource Locator
UTR	Unique Transaction Reference (number)
Admin	Administrator
Firebase Firestore	Google backend platform / Cloud Firestore NoSQL document database

ABSTRACT

Traditional college mess and local tiffin services frequently depend on manual menu communication, subscription records, meal-count estimation, billing notebooks, phone calls and repeated coordination between students and service providers. These practices can create inaccurate records, communication gaps, difficulty in managing skipped meals, and avoidable food preparation. MESSIFY is proposed as a centralized web-based mess and tiffin management system that connects students, mess owners, delivery partners and administrators through role-based digital workflows.

The implemented system provides mess discovery, subscription management, daily menu visibility, meal skipping, billing and ledger management, manual UPI payment verification, delivery-route management, QR-based tiffin identification, notifications and an AI-assisted user-support component. The application is implemented using React.js for the user interface and Firebase Authentication and Cloud Firestore for authentication and cloud data management. QR functionality is supported through QR libraries, while delivery functions maintain routes, stops and status information.

The current AI functionality is intentionally lightweight and rule-based. It uses user context, meal timing, active subscriptions and available menu information to generate meal suggestions, while a rulebased assistant answers common questions and can navigate users to relevant pages. The implementation therefore demonstrates AI-assisted personalization without claiming a trained machine-learning model. Advanced demand prediction, food-waste prediction, collaborative recommendation and delivery optimization are identified as future extensions.

CHAPTER 1 – INTRODUCTION

The food-service ecosystem has undergone substantial digital transformation through online ordering, digital payments, restaurant discovery, delivery tracking and recommendation systems. However, small and medium-sized mess and tiffin providers often continue to manage recurring meal services using informal communication and manually maintained records. Students living in hostels, payingguest accommodation, rented apartments and working professionals may depend on recurring meal plans rather than one-time restaurant orders.

MESSIFY addresses this specific recurring-service problem. Instead of treating every meal as an isolated food order, the platform models a mess or tiffin service around subscriptions, daily menus, meal skipping, payment verification, recurring billing, delivery coordination and tiffin identification. This distinction is important because a student may subscribe for a week or month, skip selected meals, receive a recurring delivery and require a consolidated ledger rather than a conventional restaurant checkout.

The system provides separate role-based experiences. Students can discover messes, view menus, subscribe, submit payment proof, skip meals, view billing and track deliveries. Mess owners can manage menus, subscribers, payments, earnings, custom requests and delivery partners. Delivery partners receive assigned routes, can advance delivery stops and scan QR codes associated with tiffin boxes. Administrators have an overview of users, messes, bookings, payments and reports.

The central objective is not merely to digitize existing records, but to connect the major operational actors in one workflow so that information moves consistently between them. The research paper supplied for this project similarly describes MESSIFY as a centralized platform for students and local tiffin or mess providers.

1.1 PROBLEM STATEMENT, OBJECTIVES AND SCOPE

From a system perspective, the problem is not limited to the absence of a mobile or web interface. The larger issue is the absence of a common information flow between the people involved in a recurring meal service. A student generates information about subscription, payment, menu preference and skipped meals; the mess owner uses that information to prepare meals, maintain records and coordinate delivery; and the delivery partner requires customer, address and route information. When these activities are handled separately through calls, messages or notebooks, the same information may need to be communicated several times. MESSIFY addresses this fragmentation by creating a shared digital workflow in which the relevant record is created once and then becomes available to the appropriate role.

The scope of the academic prototype is deliberately defined around the features that are implemented in the supplied project. It includes authentication, role-based access, mess discovery, subscription handling, menu management, meal skipping, payment-proof submission and approval, ledger information, delivery routes, QR-based tiffin identification, notifications and rule-based AI assistance. The project does not claim production-grade payment settlement, advanced machine-learning prediction or fully optimized navigation. This boundary is important because it keeps the thesis aligned with the demonstrated implementation and provides a clear foundation for future enhancement.

The main problem is the fragmented and manual management of recurring mess services. Students may need to contact a provider to know the daily menu, request a skipped meal, confirm payment status or ask about delivery. Mess owners must maintain subscriber lists, determine approximate meal quantities, record skips, verify payments and coordinate deliveries. As the number of subscribers grows, these activities become time-consuming and more vulnerable to human error.

A second problem is flexibility. Students do not necessarily require every meal every day. Without a structured skip-meal mechanism, the service provider may prepare unnecessary meals and billing records may become difficult to reconcile. A third problem concerns delivery visibility and tiffin identification. Multiple deliveries to different locations can lead to confusion unless routes, stops and container identities are digitally tracked.

Project Objectives :

1. Create a centralized platform connecting students, mess owners, delivery partners and administrators.

2. Allow students to discover mess services and manage subscriptions digitally.
3. Create a centralized platform connecting students, mess owners, delivery partners and administrators.
4. Maintain digital payment records, billing information and ledger entries.
5. Support delivery routes, stops and status updates.
6. Use QR identification to associate reusable tiffin boxes with users and messes.
7. Provide notifications for important subscription, payment and delivery events 8. Provide rulebased AI assistance and meal suggestions using available system data.
9. Reduce manual coordination and communication gaps.
10. Provide a foundation for future data-driven demand and waste prediction

1.2 BACKGROUND AND MOTIVATION

The motivation for MESSIFY comes from the recurring nature of mess services. In a restaurant ordering model, a customer generally makes an individual order and the transaction ends after delivery. In a mess or tiffin model, however, the relationship continues for days or months. A customer may have an active subscription, receive a daily menu, skip selected meals, make periodic payments and repeatedly receive a tiffin at a known location. This makes the service closer to a subscription management problem combined with food delivery than to a simple restaurant-ordering problem.

The project also considers the operational viewpoint of a small mess provider. Such providers need to know how many subscribers are active, which meals have been skipped, which payments are pending, which menu should be displayed and which tiffins have to be delivered. A centralized application can make these records easier to view and update. The motivation is therefore practical as well as technical: the project demonstrates how a common software platform can organize a real-world service without requiring the provider to maintain multiple disconnected records.

The motivation for MESSIFY comes from the recurring nature of mess services. In a restaurant transaction, a customer usually selects a meal, pays and receives an order. A mess relationship is different: the student may commit to a plan, the provider prepares meals repeatedly, and both sides need a shared understanding of menus, skips, payments and delivery status.

The research paper identifies common practical problems including unclear menu information, difficulty communicating meal requirements, manual bill calculation, difficulty recording skipped meals and dependence on phone calls, notebooks, spreadsheets or verbal communication. These observations form the functional motivation for the implemented modules.

Another motivation is operational transparency. A digital platform can create a traceable record for a subscription, a payment submission, a skip request, a billing entry and a delivery event. This does not automatically guarantee correctness, but it provides a structured foundation on which correctness can be checked and improved.

The project also explores how lightweight AI can improve usability without overstating the maturity of the technology. The current source code contains a rules-based meal suggestion generator that selects a meal slot from the time of day and uses active subscription and menu information. The assistant component uses predefined keyword rules to answer common questions and navigate users to relevant

screens. This approach is appropriate for an academic prototype because it demonstrates intelligent interaction while remaining transparent about the absence of a trained model.

The overall motivation can therefore be summarized as: digitize recurring mess operations, reduce communication overhead, improve meal flexibility, improve delivery traceability, and create a foundation for intelligent planning.

CHAPTER 2 – LITERATURE SURVEY

The literature and related-system analysis for MESSIFY can be understood through four connected areas: digital food ordering, subscription and recurring-service management, intelligent food recommendation, and identification or tracking technologies. Existing food-delivery systems demonstrate the usefulness of digital menus, ordering, payments and delivery status, while recommendation research demonstrates how food choices can be personalized using user preferences and contextual information. MESSIFY combines ideas from these areas for a more specific domain in which meals are provided repeatedly under a mess or tiffin subscription.

The supplied research paper identifies the distinction between conventional food-delivery platforms and mess-oriented services. Generic food-ordering applications are primarily designed around individual orders, whereas a mess service requires recurring subscriptions, meal skipping, repeated billing and regular delivery. This distinction motivates the domain-specific design of MESSIFY. The project does not attempt to replace large restaurant platforms; instead, it focuses on a smaller but operationally different category of food service.

The literature and related work supplied with the project paper identifies several technology areas relevant to MESSIFY: online food ordering and delivery, meal skipping and food-waste reduction, QRbased identification and tracking, and AI-assisted food management. The proposed system combines these areas around the recurring mess/tiffin use case rather than treating them as separate applications.

Online food-delivery platforms have demonstrated the value of connecting customers, food providers and delivery personnel through a shared digital interface. Typical capabilities include restaurant discovery, ordering, payment and delivery tracking. However, the supplied research paper observes that such systems are primarily optimized for individual food orders and do not focus on recurring monthly mess plans, skipped meals and student-oriented tiffin subscriptions.

Meal skipping is particularly relevant to recurring food services because preparation quantities are influenced by expected attendance. The project paper identifies the mismatch between prepared meals and actual consumption as a source of food waste and highlights the absence of a convenient skip mechanism in traditional systems.

QR technology is widely used for identification, verification and tracking. In MESSIFY, the same concept is applied to reusable tiffin containers. The objective is operational identification: a delivery

worker can associate a physical container with a student and mess record, reducing confusion during distribution.

AI-based recommendation and assistance systems are also relevant. The paper cites recent work on personalized meal recommendations and health-aware food recommendation. These studies motivate future extensions, but the current MESSIFY implementation uses rule-based assistance rather than a trained machine-learning model.

2.1 COMPARATIVE ANALYSIS

A review of the project domain indicates that many digital food systems concentrate on restaurant discovery, individual ordering, online payment and delivery. These capabilities are useful but do not by themselves solve the recurring administrative requirements of a mess. A monthly subscriber may need to skip a lunch without cancelling the entire subscription, while the provider needs that skip information for meal preparation and billing records. Similarly, the provider may need to approve a payment proof and then activate the associated subscription. These interactions require a workflow that is more persistent than a one-time order.

The research gap addressed by MESSIFY is therefore the integration of recurring mess operations into a single role-based platform. The contribution of the project is primarily system-level integration: student services, provider operations, delivery activities, QR identification, notifications and lightweight intelligent assistance are connected through a common cloud-backed data model. The present implementation provides the foundation on which more advanced prediction and recommendation models can later be developed.

Table 2.1 Comparison table of Different App

Capability	Traditional Mess	Food Delivery Platform	MESSIFY
Recurring subscription	Manual / informal	Limited	Core feature
Daily menu visibility	Phone / message	Yes	Yes
Meal skipping	Manual request	Usually order cancellation	Dedicated skip workflow
Billing ledger	Notebook / spreadsheet	Transaction based	Subscription ledger
Mess owner dashboard	No / basic	Restaurant dashboard	Dedicated dashboard
Delivery route management	Manual	Advanced in many systems	Route + stop tracking

Reusable tiffin identification	Manual	Not central	QR-based
Rule-based AI assistance	No	Varies	Implemented
Demand prediction	Manual estimation	Varies	Future scope
Student-focused mess discovery	Local word-of-mouth	Restaurant-centric	Core feature

The principal research gap addressed by the project is the lack of a single lightweight platform designed specifically for recurring mess and tiffin operations. The contribution is therefore integrative rather than claiming a new machine-learning algorithm. MESSIFY combines subscription management, meal flexibility, digital billing, delivery coordination, QR identification and AI-assisted support in a single workflow.

This distinction is important for academic evaluation. The system should be assessed as a software engineering and applied intelligent-system project rather than as a fully trained AI research model. Its novelty lies in domain integration and workflow design, while future research can introduce predictive models once sufficient historical meal, skip, menu and delivery data are collected.

2.2 REVIEW OF RELATED WORK

The literature survey for MESSIFY: AI-Based Smart Mess Management System reviews existing approaches to online food ordering, mess management, meal subscription systems, recommendation systems, and AI-based assistance. These systems provide useful ideas for improving food service management, but there is still scope for a unified platform that combines menu management, meal skipping, subscriptions, order handling, and intelligent user assistance.

The following review is written in a thesis-friendly academic style. It discusses the main areas of related research and identifies how MESSIFY addresses the needs of students and mess providers.

Study 1: Online Food Delivery Research — A Systematic Literature Review

This study, published in the International Journal of Contemporary Hospitality Management in 2022, reviewed 43 research articles on online food delivery published between 2015 and March 2021. It examined the development of online food delivery services and the different challenges faced by customers, restaurants, and delivery platforms. The study provides a useful foundation for understanding digital food service management. However, MESSIFY focuses more specifically on recurring mess subscriptions, meal skipping, and coordination between students and mess providers.

ScienceDirect

Study 2: Application of Web Technologies in Food Ordering Systems — A Systematic Literature Review

A systematic literature review published in 2025 examined web-based food ordering systems and the technologies used to develop them. It discussed how web applications can support online ordering, improve accessibility, and simplify food service operations. The review also highlighted challenges associated with security, scalability, and user experience. MESSIFY follows a similar digital approach while focusing on the specific requirements of mess and tiffin services, including menu visibility, subscription management, and meal-related requests.

Study 3: A Survey on Online Food Ordering System

Jadhav et al. presented a survey of online food ordering systems in 2025, discussing a web application architecture based on the MERN stack. The study covered important features such as user authentication, menu management, order processing, and payment integration. These functions are relevant to the development of a centralized food ordering platform. MESSIFY extends this general concept towards

mess management by combining student subscriptions, mess-owner operations, and basic intelligent assistance in one application. IJCRT

Study 4: Web-Based Food Ordering and Intelligent Recommendations

Recent literature on food ordering systems has also considered the use of artificial intelligence for personalized menus and automated recommendations. These approaches demonstrate how intelligent features can improve the user experience by helping users find suitable food options. In MESSIFY, a basic rule-based assistant and context-based meal suggestions support common user queries and meal discovery. Unlike systems that use trained machine-learning models, these features provide limited, ruledriven assistance rather than advanced predictive intelligence.

2.3 RESEARCH GAP

The reviewed studies show that online food ordering platforms can simplify menu browsing, order placement, and restaurant operations. However, a general food delivery system does not necessarily address the recurring needs of college students and local mess providers. These needs include managing meal subscriptions, viewing daily menus, requesting meal skips, maintaining billing records, and communicating with mess owners.

MESSIFY addresses this application-specific gap by providing a centralized platform for students and mess providers, together with basic AI-style assistance. Its aim is to make routine mess operations more organized and accessible. Advanced features such as machine-learning-based demand forecasting and food-waste prediction can be considered for future development.

References

Use the following references in your thesis bibliography and verify the final citation format required by your college.

[1] “Online food delivery research: a systematic literature review,” International Journal of Contemporary Hospitality Management, vol. 34, no. 8, pp. 2852–2883, 2022.

.[2] N. Nurhinaya et al., “Penerapan Teknologi Web dalam Sistem Pemesanan Makanan (Food Order System): Systematic Literature Review,” Jurnal Teknologi dan Bisnis Cerdas, vol. 1, no. 3, 2025.

.[3] P. Jadhav, K. Mahajan, S. Nagare, S. Kumbhar, and A. Bhamre, “A Survey on Online Food Ordering System,” International Journal of Creative Research Thoughts, vol. 13, no. 4, pp. f598–f603, 2025.

CHAPTER 3. METHODOLOGY

The development methodology followed a requirement-to-module approach. First, the recurring activities of students, mess owners, delivery partners and administrators were identified. These activities were converted into functional requirements such as authentication, mess discovery, subscriptions, menus, skipping, payments, billing, delivery, QR scanning and notifications. The requirements were then mapped to role-specific interfaces so that each user sees the operations relevant to that role.

The implementation was organized as a web application with a cloud-backed data layer. React components provide the user interface, routing separates the major application areas, Firebase Authentication manages identity, and Cloud Firestore stores application records. Supporting libraries provide QR generation and scanning, while utility and context modules help separate shared application behavior from individual pages. This structure makes the prototype easier to demonstrate and provides clear extension points for future development.

The project follows a structured software development methodology consisting of requirement identification, role and workflow design, interface development, cloud integration, module integration, testing and refinement. The methodology is practical and iterative: individual features were implemented and then connected through shared Firebase data-access functions.

Phase 1 – Requirement identification: The project first identifies the daily operational problems of students, mess owners and delivery personnel. These are translated into requirements for authentication, mess discovery, subscriptions, menus, meal skipping, payments, billing, delivery, QR tracking, notifications and AI assistance.

Phase 2 – Architecture and interface design: The application is divided into role-specific pages and shared components. React Router provides navigation between pages, while common UI components provide consistent visual structure. The system is designed around four principal roles: student/customer, mess owner, delivery partner and administrator.

Phase 3 – Backend integration: Firebase Authentication is used for login and registration. Cloud Firestore is used for application data including user profiles, mess profiles, menus, subscriptions, skips, payments, routes and notifications. Firebase Storage is used for uploaded images such as payment proof where configured.

Phase 4 – Functional integration: Modules are connected through centralized Firestore helper functions. For example, subscription creation writes a subscription record, payment submission creates a payment

record, approval changes payment and subscription states, and meal skipping creates a skip record that can be read by the mess dashboard.

Phase 5 – Testing and refinement: Authentication, role-based navigation, menus, subscriptions, skips, billing, payment approval, route handling, QR operations and AI assistance are tested as individual workflows and as an integrated system. The supplied research paper confirms that the completed system was tested for functionality, data flow, authentication, payments, meal management, QR operations, delivery features and user interaction.

3.1 – REQUIREMENTS AND SYSTEM DESIGN

The system requirements were derived from the project source code and the functional description in the supplied research paper. The design follows a role-based architecture so that each user sees the operations appropriate to that role.

The functional requirements describe what the system must allow each actor to accomplish. Students require registration and login, mess discovery, menu viewing, subscription selection, payment submission, meal skipping, billing visibility and delivery status. Mess owners require menu and subscriber management, payment approval, ledger information, delivery assignment, QR-related operations and handling of custom requests. Delivery partners require access to assigned routes and QR scanning, while administrators require a supervisory view of users, messes, bookings and payments.

The non-functional requirements are equally important for a practical web system. The interface should be understandable to users who may not have technical knowledge, data should remain consistent across role-based workflows, and unauthorized users should not be allowed to perform owner or administrator operations. Because the application depends on cloud services, network availability and correct configuration are also relevant operational requirements. These requirements guided the design of the prototype and the testing of its major workflows.

Table 3.1 Functional Requirement Table

ID	Functional Requirement	Primary User
FR-01	Register and authenticate users with role information.	All
FR-02	Create and maintain public mess profiles.	Mess Owner
FR-03	Browse messes and view service details.	Student
FR-04	Create subscriptions for lunch, dinner or both.	Student
FR-05	Submit payment information and proof for verification.	Student

FR-06	Approve or reject submitted payments.	Mess Owner
FR-07	Create and update daily lunch and dinner menus.	Mess Owner
FR-08	Record skipped meals for a subscription and date.	Student
FR-09	Maintain monthly ledger entries for billing.	Student / Mess
FR-10	Create delivery routes and manage stops.	Mess / Delivery
FR-11	Scan and update tiffin-box QR status.	Delivery
FR-12	Send and display notifications.	System
FR-13	Provide AI-assisted help and meal suggestions.	Student
FR-14	Provide administrative overview of users, messes and payments.	Admin

Non-functional requirements include usability, maintainability, access control, data consistency, responsive interface behavior and reasonable cloud-backed availability. Since the application is a prototype, production-level scalability, automated payment settlement and large-scale predictive accuracy are not claimed.

3.2 SYSTEM ARCHITECTURE

The architecture follows a layered organization so that user interaction, application logic, data storage and external services have distinguishable responsibilities. At the user layer, students, mess owners, delivery partners and administrators interact with role-specific screens. The application layer coordinates pages and reusable components. The backend and database layer provides authentication and cloud data storage. External services support functions such as QR processing, mapping or image storage where configured.

A typical interaction begins when an authenticated user performs an action such as opening a menu, submitting a payment proof or recording a meal skip. The application identifies the user's role and relevant record, communicates with the Firebase-backed data layer, and updates the appropriate document. Other authorized users can then see the resulting state through their own dashboards. This flow is particularly important for recurring operations because the same business event can affect more than one actor.

The MESSIFY architecture consists of four major layers: User Layer, Application Layer, Backend and Database Layer, and External Services Layer. This layered structure separates user interaction from business workflows and persistent data.

Figure: MESSIFY System Architecture

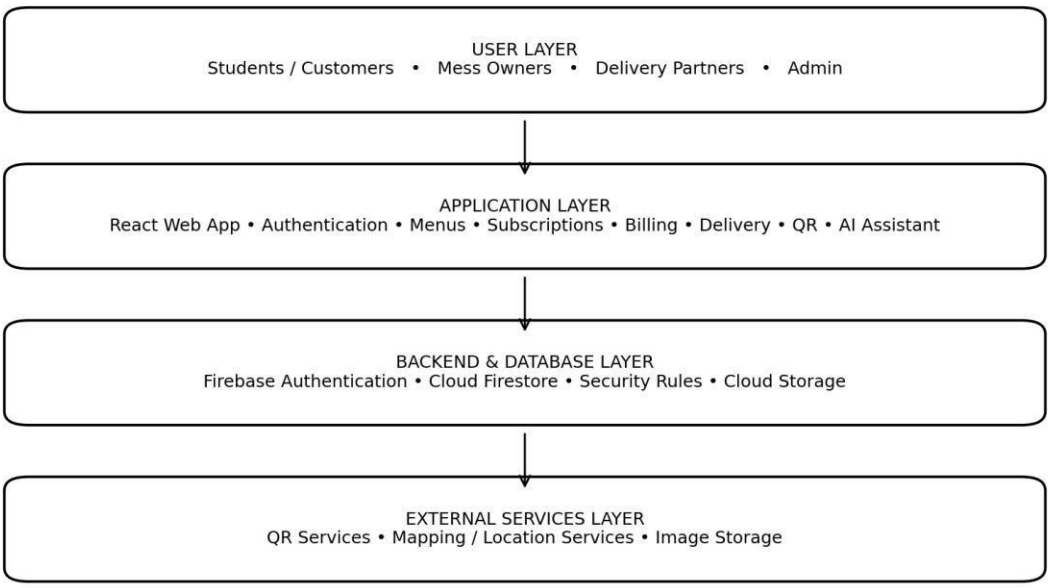


Figure 3.1 – MESSIFY System Architecture

User Layer: Four principal actors interact with the platform: students/customers, mess owners, delivery personnel and administrators. Application Layer: React.js provides the browser-based interface and rolespecific modules. Backend and Database Layer: Firebase Authentication manages identity while Cloud Firestore stores operational records. External Services Layer: QR libraries, mapping/location functionality and cloud image storage support specialized functions.

The architecture supports a request-response and event-driven pattern. A user action such as subscribing or skipping a meal invokes a Firebase helper function. The helper reads or writes the relevant Firestore document. For route and menu data, real-time listeners are available in the source code so interface state can react to changes without requiring a complete page reload.

The supplied paper describes the same four-layer architecture and identifies Firebase Authentication, Cloud Firestore, security rules, QR services, mapping services and cloud-based image storage as core components.

3.3 USER WORKFLOW

The student workflow begins with authentication and continues through mess discovery, menu inspection and subscription. After selecting a service, the student can submit payment information and wait for provider verification. Once the subscription is active, the student can view meal information and record a lunch or dinner skip when necessary. Billing and delivery information provide additional visibility into the ongoing service. The AI-assisted component is presented within this workflow so that recommendations and common service guidance are available without creating a separate application.

The mess-owner workflow is centered on daily operations. The owner manages menu information, reviews active subscribers, monitors payment submissions and observes meal skips. Delivery-related information can be organized into routes and stops, while QR-linked tiffin records provide an additional identification mechanism. The delivery partner then works through assigned stops and can scan the tiffin identifier. Administration remains a supervisory role for reviewing platform-level records.

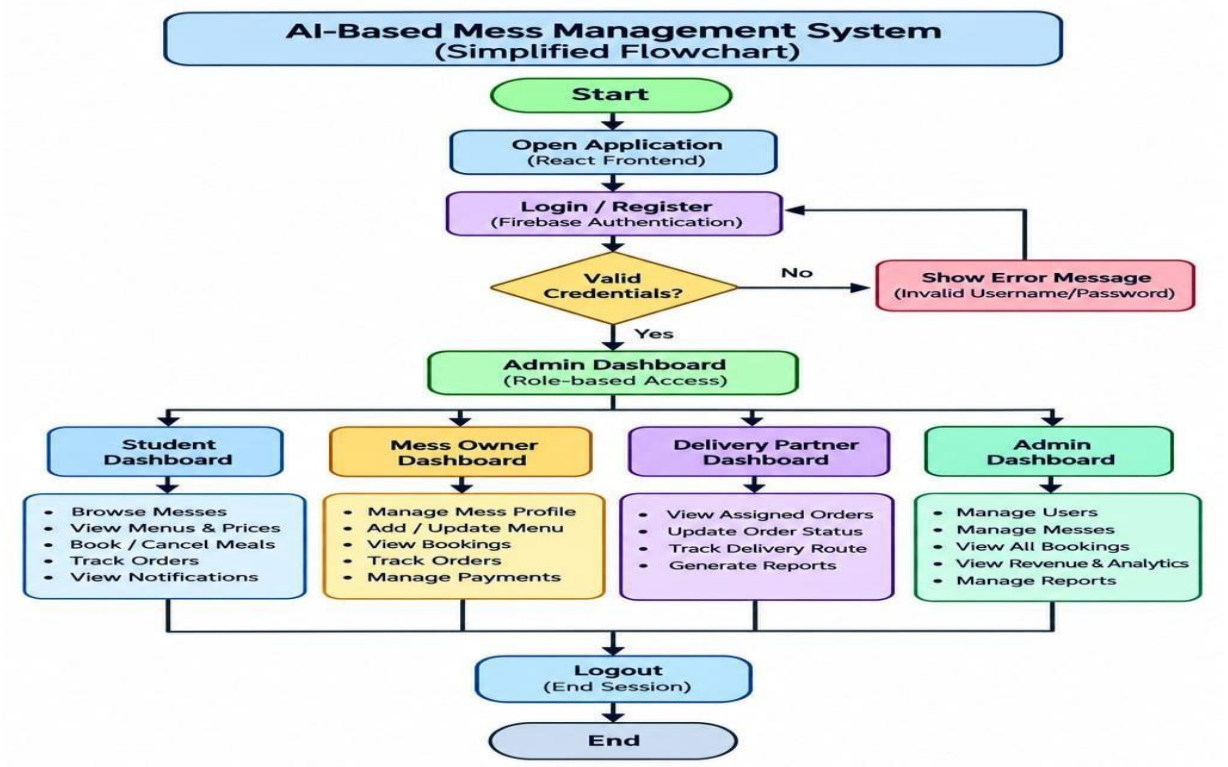


Figure 3.2 – Primary Student-to-Delivery Workflow

Student workflow: The student registers or logs in, browses available messes, selects a provider, chooses a subscription plan and submits payment proof where required. After approval, the subscription becomes

active. The student can view the current menu, skip a lunch or dinner when necessary, inspect billing records and follow delivery information.

Mess-owner workflow: The owner creates or maintains a public mess profile, updates the daily menu, views active subscribers, receives payment submissions, approves or rejects payments, reviews meal skips, maintains ledger information and coordinates delivery routes and partners.

Delivery workflow: A delivery partner receives assigned routes and stops. The route record maintains a current stop index and status. The partner can advance the route and scan a QR identifier associated with a reusable tiffin box. The scan updates the box status and timestamp.

Administrative workflow: The administrator has an overview of system-level records such as users, messes, subscriptions/bookings and payments. This provides a supervisory layer over the service.

3.4 DATABASE DESIGN

The database design is based on Cloud Firestore's document-oriented structure. Records are separated according to the application's business concepts, while identifiers such as studentId, messId, subscriptionId and route-related identifiers connect related records. This approach is suitable for the prototype because the major operations generally retrieve records associated with the currently authenticated user, selected mess or active route. It also allows different modules to update their own operational records without requiring a traditional relational join at the application interface.

The principal data concepts represented by the project include user profiles, mess profiles, menus, subscriptions or bookings, skipped meals, orders, payments, ledger entries, routes, tiffin boxes and notifications. Maintaining explicit status fields is useful because many operations move through states—for example, a payment can remain pending before approval, and a subscription can move between relevant active or inactive states. The data model therefore supports both storage and workflow coordination.

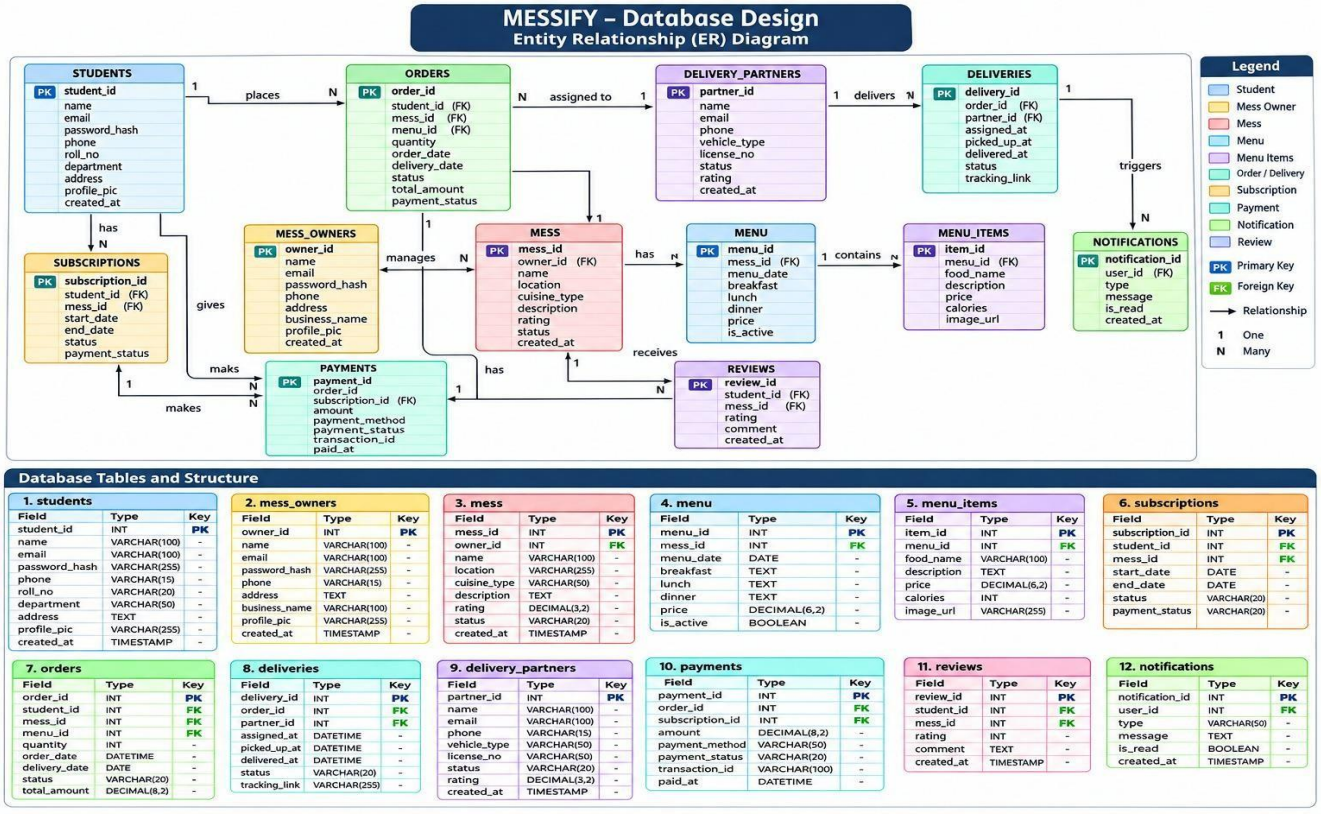


Figure 3.3 – Firestore Logical Data Model Diagram

Table 3.2 Overview of Database Collection

Collection	Purpose	Representative Fields
users	Identity and role profile	uid, name, email, role, phone, area
messProfiles	Public mess listing	ownerId, messName, area,
		rating, subscriberCount
menus	Date-specific meal menu	messId, date, lunch, dinner
subscriptions	Recurring customer plan	studentId, messId, plan, status, startDate
skips	Meal skip events	subscriptionId, studentId, messId, date, meal
ledger	Monthly billing history	studentId, messId, month, entries
payments	Payment submissions and review	amount, UTR, screenshotUrl, status
routes	Delivery route and stops	deliveryPersonId, date, meal, stops, CurrentStopIndex
tiffinBoxes	Reusable box identity	qrId, studentId, messId, status
notifications	User alerts	userId, title, message, read, createdAt

Cloud Firestore uses document-oriented records. Composite business relationships are represented by identifiers such as studentId, messId and subscriptionId. This keeps the implementation straightforward for a prototype and supports targeted queries. The source code centralizes data access in firestore.js, which reduces duplication across pages and makes future backend changes easier.

The research paper also identifies users, mess profiles, menus, subscriptions, skips, orders, routes, ledger, tiffin boxes, payments and notifications as the principal data concepts used by the system.

3.5 FUNCTIONAL MODULES

The MESSIFY system consists of several functional modules that work together to simplify mess management. The Student Module allows students to register, log in, view menus, manage meal subscriptions, skip meals, place orders, and check payment details. The Mess Owner Module helps mess owners manage daily menus, meal plans, subscriptions, orders, and billing records.

The AI Assistance Module provides a rule-based chatbot to answer common user queries and offers context-based meal suggestions. The Payment and Billing Module maintains payment and subscriptionrelated information. The Delivery Module supports order and delivery status management where implemented. Finally, the Database Module stores and manages important information, including user details, menus, subscriptions, orders, and payment records. Together, these modules provide a centralized platform for managing mess-related activities efficiently.

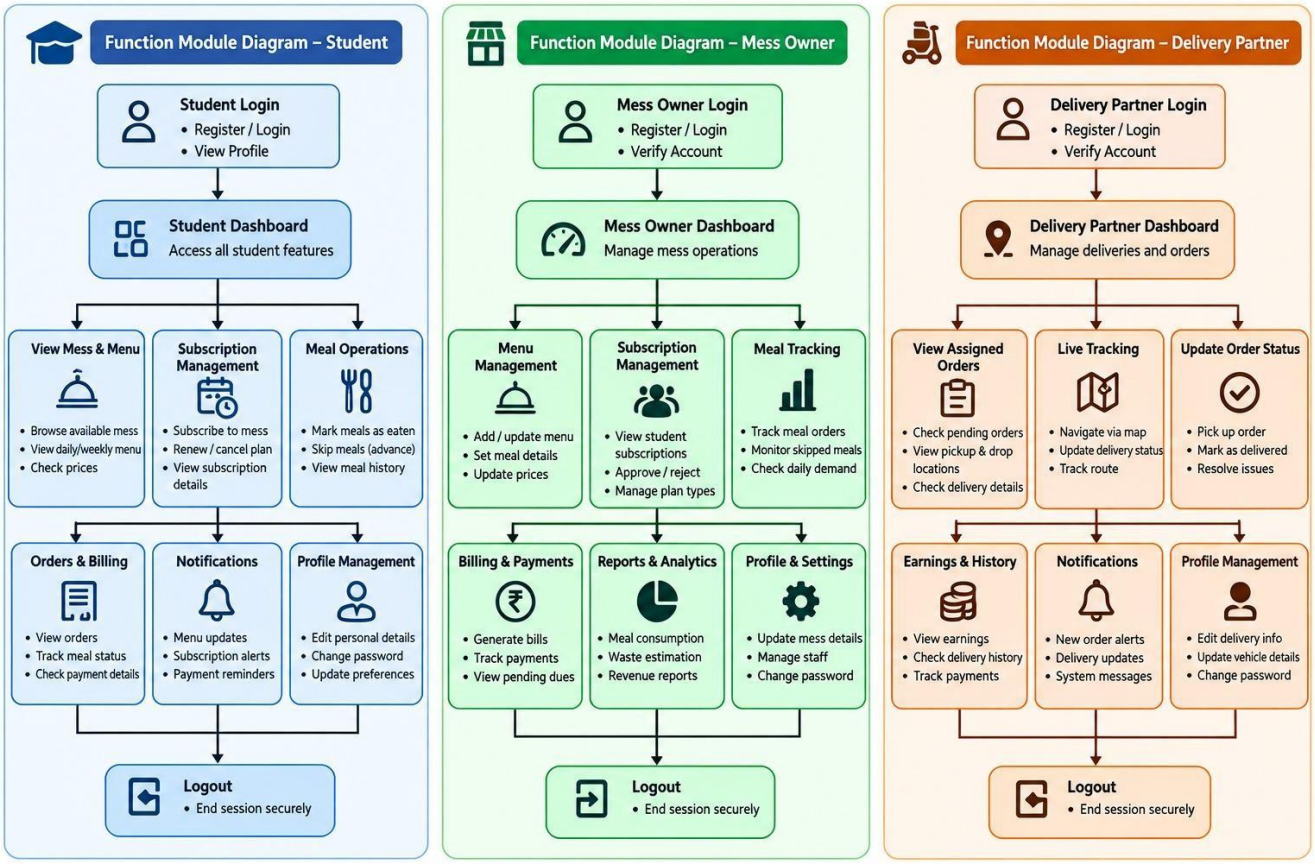


Figure 3.4 – Functional Module Diagram

3.6 STUDENT

The student module is designed to reduce repeated communication with the mess provider. After signing in, the student can access information that would otherwise be requested through messages or calls. The dashboard combines current subscription information with meal-related actions, while individual pages provide more detailed operations. The design is particularly focused on recurring use, because the student is expected to return to the platform regularly rather than use it only once.

The meal-skipping feature is one of the most domain-specific functions. Instead of asking a student to cancel or modify an entire subscription, the system records the specific meal that will not be consumed. This creates a digital event that can be considered by the mess owner while planning preparation.

Similarly, the billing and payment workflow maintains a visible history rather than depending entirely on informal confirmation.

1. Authentication and Profile: Students register and sign in using Firebase Authentication. Profile data includes identity, contact information, role and area information used by the application.

2. Browse Mess: Students can view available mess profiles and select a provider. The recommendation logic can prioritize messes using rating and area matching when profile information is available.

3. Subscription Management: Students select a plan such as lunch, dinner or both. The subscription record stores the student, mess, plan, status and start date.

4. Payment Submission: The payment workflow is based on direct UPI payment to the mess owner. The student submits payer details, UTR number and payment proof. The payment remains pending until the mess owner reviews it.

5. Daily Menu and Meal Skipping: Active subscriptions display the applicable meal menu. A student can submit a skip record for lunch or dinner. This gives the mess owner a structured signal for meal preparation and billing.

6.Billing and Ledger: The billing page reads ledger entries and payment history to provide a consolidated view of charges and transaction information.

7. Delivery Tracking: Students can access delivery-related status information when a route has been created and activated by the mess.

8. AI Assistance: The student dashboard presents a personalized meal suggestion based on current meal slot, active subscription and available menu information. The assistant can answer common service questions and deep-link to relevant pages.

3.7 MESS OWNER

The mess-owner module converts the provider's daily administrative activities into a structured dashboard. Menu management controls what students see for the selected date, while subscriber management provides a list of users associated with the mess. Payment approvals create a clear checkpoint between a student's payment submission and activation of the subscription. This reduces ambiguity because the payment record contains supporting details such as UTR information and proof where configured.

The owner also receives information that is directly relevant to operational planning. Meal skips provide a signal about expected meal consumption, while route and tiffin records support delivery coordination. Custom requests create a structured alternative to informal messages. Together, these modules give the owner a central place to perform tasks that would otherwise be distributed across notebooks, messaging applications and separate lists.

The mess-owner dashboard is designed around the daily operating cycle of a tiffin provider. The dashboard summarizes active subscribers, earnings, lunch skips, dinner skips and the current menu. Pending payments are highlighted so that a provider can review payment evidence before activating a subscription.

Menu Manager: Owners can update lunch and dinner menu items and prices for the current date. Menu records are stored by mess and date, allowing the student dashboard to retrieve the appropriate daily information.

Subscriber Management: The owner can view active subscribers associated with the mess. Subscription status is used to distinguish active users from pending, paused or rejected states.

Payment Approvals: Payment submissions contain amount, payer name, UPI information, UTR number and screenshot URL where configured. The owner can approve or reject the submission.
Approval activates the subscription and records the applicable payment split.

Mess Ledger: The ledger module provides a structured view of monthly meal-related financial entries. The source code represents entries with date, meal, amount and type such as charge or skip.

Delivery Partner Management: Owners can associate delivery users with the mess and use them for route assignment. Delivery routes contain stops with student identity, address, coordinates and delivery status.

QR Settings and Tiffin Management: The owner can configure QR-linked tiffin identification so that reusable containers can be tracked through scan events.

3.8 DELIVERY AND ADMINISTRATION

The delivery module provides a lightweight digital representation of the delivery process. A route contains a date, delivery person and an ordered collection of stops. The delivery partner can view the assigned sequence and update progress as stops are handled. This is intentionally described as route and stop management rather than a complete navigation engine, because the current project does not implement a full optimization algorithm.

QR scanning connects the physical tiffin container with the digital system. A successful scan can update the corresponding tiffin record with relevant identity, mess, status and scan-time information. The administrator module provides a broader supervisory perspective, allowing system-level records such as users, messes, bookings and payments to be reviewed. Notifications provide another communication mechanism for important application events.

Delivery Dashboard: Delivery users see assigned routes for the selected date and can work through their stops. The route record contains the mess, delivery person, date, meal, list of stops, current stop index and route status.

Route View: The route view provides a sequence of delivery stops and supports advancing the current stop. This creates a simple operational representation of delivery progress without claiming a full vehicle-routing optimization algorithm.

QR Scanner: The Scan QR module uses the html5-qrcode library to read QR identifiers. A successful scan updates the corresponding tiffinBoxes record with the student, mess, status and scan timestamp. The mechanism is intended to reduce container mix-ups.

Administration: The administrator dashboard provides a system-wide view of users, mess profiles, subscriptions/bookings and payments. This role is useful for monitoring the platform rather than performing the day-to-day activities of individual mess owners.

Notifications: The system creates notifications for events such as payment submissions and can display them through the notification component. Users can mark individual notifications or all visible notifications as read.

The research paper identifies delivery personnel and administrators as distinct actors and describes route management, QR identification, notifications and administrative oversight as part of the proposed system.

3.9 AI ASSISTANCE AND RECOMMENDATION

The intelligent component of MESSIFY is designed around context available inside the application. The meal suggestion logic considers the current time to determine a meal slot and then looks for relevant subscription and menu information. This means the system can provide a contextual suggestion without requiring a large external dataset. The assistant similarly uses predefined keyword groups to recognize common service questions related to subscriptions, payments, skipping, billing, delivery and support.

The project also contains a simple recommendation score for available messes. Rating information contributes to the score, and area matching can provide an additional signal when the relevant profile data is available. These mechanisms demonstrate the idea of personalization while remaining transparent and deterministic. Because the current source contains no trained model, dataset, training pipeline or predictive accuracy evaluation, the thesis treats advanced machine learning as future work rather than presenting the existing rules as a learned AI model.

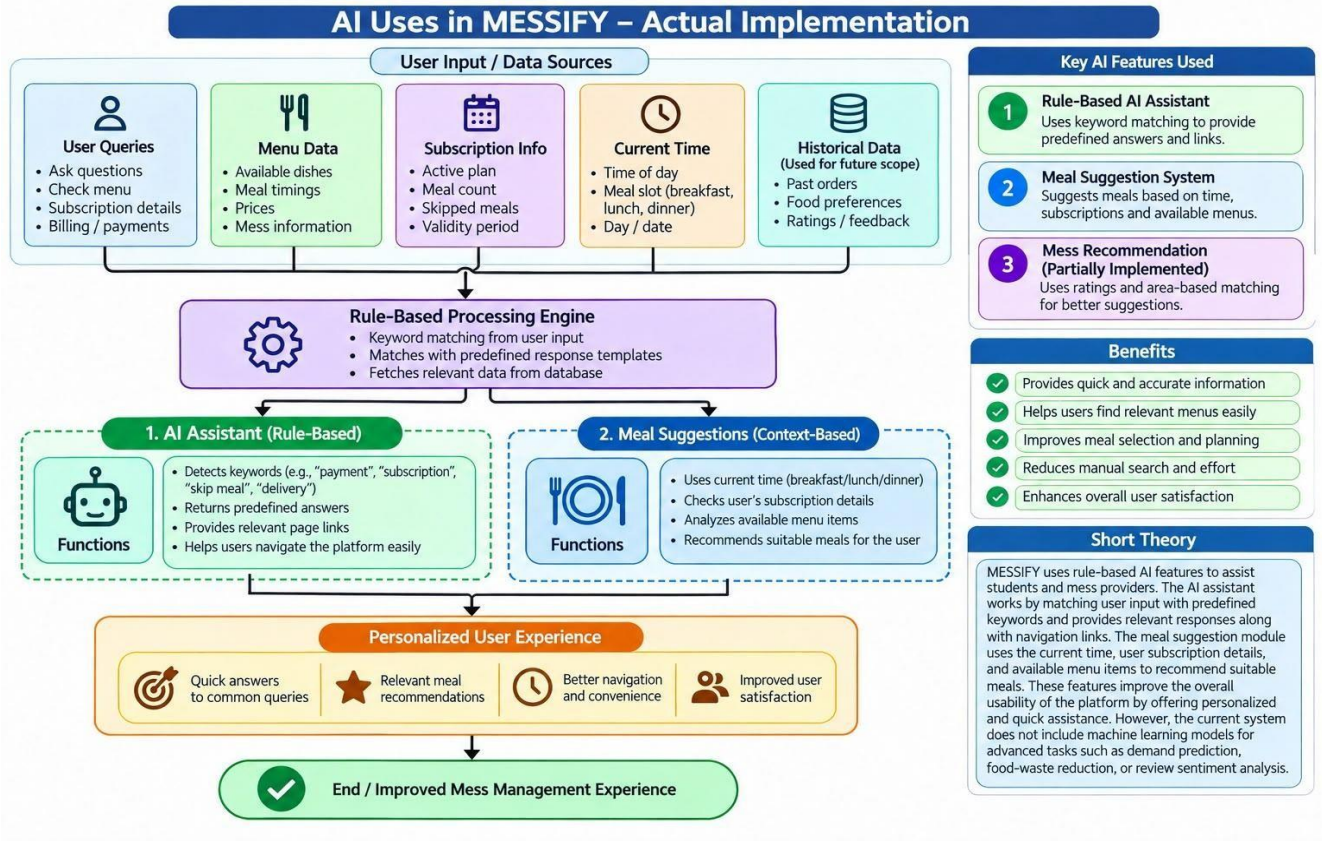


Figure 3.5 – Current AI Assistance and Personalization Flow

The project uses the term AI-assisted because intelligent behavior is implemented through deterministic rules and personalization logic. The source code explicitly documents the meal suggestion generator as a lightweight, rules-based personalization mechanism rather than a real machine-learning model.

Meal suggestion logic: The system determines a meal slot from the current time: breakfast before 11:00, lunch before 17:00, and dinner after that. It searches active subscriptions for a mess with menu information for the relevant slot, extracts one or two menu items, and generates a contextual suggestion sentence. If no relevant subscription or menu is available, a fallback message guides the student toward subscribing or browsing a mess.

AI assistant logic: The assistant contains keyword groups for common intents such as subscription, payment, skipping, billing, delivery, becoming a mess owner, commission, password and support. A matched rule returns an answer and, where appropriate, a navigation link. If no rule matches, the assistant provides a fallback response.

Recommendation logic: The student dashboard also computes a simple recommendation score for available messes. Rating contributes to the score, and an additional score can be added when the mess area matches the student's area. The highest-scoring candidates are displayed as recommendations.

Limitations of current AI: There is no trained neural network, supervised learning pipeline, model evaluation dataset or predictive accuracy metric in the current source. Therefore the thesis does not claim machine-learning performance. The supplied research paper makes the same distinction and proposes future machine-learning features such as demand prediction and personalized meal recommendations.

3.10 SECURITY CONSIDERATIONS

Security is considered at both the identity and data-access levels. Firebase Authentication provides the primary identity mechanism, while application role information determines which interface and operations are available to a user. Firestore security rules provide an additional access-control layer for stored records. In a production deployment, these controls would need to be reviewed carefully against every role and operation, particularly for payment approvals, delivery addresses and administrative functions.

The payment workflow also limits the information handled directly by the application. The current project uses UPI payment followed by submission of payer details, UTR information and proof rather than storing card credentials. Nevertheless, payment evidence and delivery information are still sensitive application data and should be protected. A production version should add stronger audit logging, formal privacy policies, retention rules, monitoring and abuse-prevention controls.

Security is important because MESSIFY handles identity information, subscription records, payment evidence, delivery details and role-specific operations. The implementation uses Firebase Authentication for identity and Firestore security rules to regulate database access.

Authentication and authorization: The AuthContext maintains the current authenticated user and profile. Role information is used to present appropriate routes and interfaces for students, mess owners, delivery partners and administrators.

Database access: Firestore rules are used to restrict and regulate access to stored data. Sensitive operations such as payment approval should remain limited to authorized mess-owner or administrative roles.

Payment proof: The system does not process card or bank credentials directly. The implemented workflow relies on UPI payment to the provider followed by submission of proof and UTR information. This reduces the scope of payment processing in the academic prototype, but payment verification remains a human-reviewed process.

Input and data integrity: Application functions use structured objects and explicit status values. Subscription states, payment states, route states and tiffin states provide predictable transitions.

Production deployment would require stronger validation, audit logging, rate limiting, secure storage policies, privacy controls and automated abuse detection.

Privacy: Delivery addresses and location-related information should only be visible to actors who need them for service fulfillment. Future production versions should define a formal privacy policy, data retention period and user-consent process.

3.11 TECHNOLOGY STACK

The supplied project package identifies React 18.3.1 and React DOM 18.3.1 for the user interface,

React Router DOM for application navigation, Firebase 10.12.2 for authentication and cloud services, Vite for development and build tooling, Tailwind CSS for interface styling, and QR-related libraries for code generation and scanning. This combination provides a practical stack for developing a responsive academic prototype with a relatively small deployment footprint.

The project is organized into pages, shared components, context, Firebase access, utilities, styles and data. This modular organization is important for maintainability because business operations can be modified without rewriting the entire interface. The supplied source also contains dummy mess data for demonstration scenarios. Such data is suitable for academic testing, but it must not be presented as real customer statistics or evidence of production-scale performance.

Table 3.3 Technology Stack and its Role in Messify

Technology	Role in MESSIFY
React.js 18	Component-based web application interface.
React Router 6	Client-side navigation and role-specific routes.
Vite 5	Development and production build tooling.
Tailwind CSS 3	Utility-based styling and responsive interface.
Firebase Authentication	User registration and authentication.
Cloud Firestore	Cloud document database for application data.
Firebase Storage	Storage for uploaded files/images where configured.
html5-qrcode	QR scanning in the delivery module.
qrcode.react	QR code generation/display.

date-fns	Date-related utilities.
JavaScript / JSX	Application logic and UI components.

The package configuration in the supplied ZIP identifies React 18.3.1, React DOM 18.3.1, React Router DOM 6.24.0, Firebase 10.12.2, html5-qrcode 2.3.8, qrcode.react 3.1.0, Tailwind CSS 3.4.19 and Vite 5.3.1 among the principal dependencies.

The project structure separates pages, shared components, context, Firebase access, utilities, styles and data. This separation supports maintainability. For example, firestore.js centralizes database operations while aiSuggestion.js isolates meal-suggestion logic and AIAgent.jsx manages the rule-based assistant interface.

The source also contains dummy mess data for situations where live mess profiles are unavailable. This is appropriate for demonstration and academic testing but should be clearly distinguished from real operational data in a production deployment.

The status column represents functional verification of the implemented project workflow for the academic prototype. It should not be interpreted as a formal statistical quality guarantee or production certification.

CHAPTER 4 - STRENGTHS AND LIMITATION

The MESSIFY system was developed as a web-based AI-assisted mess and tiffin management application to simplify daily mess-related activities through a centralized platform. The project focuses on providing useful features for students and mess owners, along with support for delivery-related operations. Different modules, including user management, menu management, meal subscriptions, meal skipping, order handling, billing, and AI-assisted services, are integrated to provide an organized mess management workflow.

The project demonstrates the use of modern web development technologies to create an interactive and user-friendly application. It includes user authentication, data management, menu and subscription handling, and basic intelligent assistance through a rule-based chatbot and context-based meal suggestions. Location-related features and QR-based tiffin identification support the management of mess services where implemented. However, the current system has certain limitations, as its AI functionality does not include advanced trained machine-learning models for demand forecasting or food-waste prediction. The effectiveness of features such as delivery tracking and payment processing also depends on their implementation and configuration. These limitations provide opportunities for future improvements and additional intelligent features.

4.1 - STRENGTHS OF THE PROJECT

A major strength of MESSIFY is its domain specificity. The project is not simply a generic foodordering interface with a different name; its workflow is organized around recurring subscriptions, meal skipping, provider approval, ledger records and regular delivery. The role-based structure is another strength because it recognizes that students, providers, delivery partners and administrators have different responsibilities and therefore should not be presented with identical controls.

Another strength is the combination of multiple technologies within one coherent academic system. React provides the front-end experience, Firebase supplies authentication and cloud storage, QR technology connects physical tiffins to digital records, and the rule-based AI layer demonstrates contextual assistance. The project therefore provides a broad demonstration of software engineering concepts while maintaining a clear boundary around features that are not yet production-ready.

- Domain-specific design: The system is designed for mess and tiffin subscriptions rather than generic restaurant ordering.
- Role-based workflow: Students, mess owners, delivery partners and administrators receive separate functional interfaces.
- Centralized cloud data: Firebase Authentication and Cloud Firestore reduce dependence on local notebooks and scattered records.
- Meal flexibility: The dedicated skip-meal workflow supports recurring service behavior and can provide better preparation signals.
- Payment verification workflow: UPI payment proof, UTR information and owner approval create a traceable academic prototype workflow.
- Delivery traceability: Routes, stops and QR-linked tiffin records address common distribution problems.
- Rule-based AI: Intelligent assistance is implemented transparently without falsely claiming a trained machine-learning model.
- Modular source structure: Shared components, Firebase helpers and utility modules support future maintenance.
- Extensibility: The data model can be expanded with historical information for future predictive analytics.

- Practical academic relevance: The project combines web development, cloud database, authentication, QR, delivery and AI-assisted features in one application.

The research paper emphasizes that the platform reduces manual work, improves communication and creates a more organized experience for students, mess owners and delivery personnel.

From a software engineering perspective, another strength is the separation of business operations into modules. This makes it possible to improve one area—such as recommendations or payment processing—without redesigning the entire user experience.

4.2- LIMITATIONS

The most important limitation is the current level of intelligence. The project uses deterministic rules rather than a trained machine-learning model. Consequently, it cannot presently learn from historical meal choices, predict demand with a measured accuracy, or automatically discover complex user preferences. The recommendation logic is also based on simple signals such as rating and area rather than learned personalization.

Operational limitations are also present. Payment remains based on UPI transfer and manual verification rather than an integrated payment gateway. Delivery is represented through routes and stops instead of full GPS navigation and optimization. The prototype depends on correct Firebase configuration and network availability, and dummy data may be used for demonstration. These limitations define the difference between the current academic prototype and a production deployment.

- The current AI assistant is rule-based and does not contain a trained machine-learning model.
- Demand prediction and food-waste prediction are proposed but are not implemented as predictive models in the current source.
- Payment processing is not an automated payment-gateway integration; the current flow relies on UPI payment and manual owner verification.
- Delivery tracking is lightweight and based on route/stop states rather than full real-time GPS navigation and optimization.
- Recommendation logic uses simple rating and area signals rather than collaborative filtering or learned personalization.
- Real-world performance depends on Firebase configuration, network availability, security rules and correct data entry.
- Prototype data may include dummy mess records and therefore should not be interpreted as real customer or operational statistics.
- Production deployment would require stronger privacy controls, audit logs, monitoring, backups, abuse protection and formal operational policies.
- Scalability and load testing at a large institutional or city-wide scale have not been established by the academic prototype.

These limitations do not invalidate the project; instead, they define the boundary between the current academic implementation and a production-ready intelligent platform. Stating these boundaries is important for a credible thesis and for identifying measurable future research directions.

CHAPTER 5 – RESULTS AND DISCUSSION

The overall result of the project is a connected web-based workflow in which the major actors of a recurring mess service can interact with shared application records. The student is no longer dependent on a separate communication channel for every routine operation, while the mess owner receives a structured view of subscriptions, menus, skips and payment submissions. Delivery-related records provide a digital representation of routes and tiffin identification, and administrative views provide platform-level oversight.

An important outcome is traceability. A subscription is represented as a digital record, a payment submission can be associated with that subscription, and an owner approval can change the resulting service state. Similarly, a skipped meal becomes a visible record instead of an informal message. This does not automatically eliminate every manual activity, but it makes the process more organized and gives the involved actors a common reference point.

The completed MESSIFY prototype demonstrates a centralized workflow for managing the main activities of a recurring mess and tiffin service. The student side brings discovery, subscription, menu, skip, billing and delivery information into one interface. The provider side brings menu management, subscriber management, payment review, earnings and delivery coordination into a corresponding dashboard.

Table 5.1 – Observed Functional Outcomes

Function	**Before Digitalization**	**MESSIFY Outcome**
Menu communication	Phone/message/manual notice	Date-based digital menu
Subscription records	Notebook/spreadsheet	Firestore subscription records
Meal skips	Verbal/message request	Structured skip event
Payment verification	Manual communication	Proof + UTR + approval workflow
Billing	Manual calculation	Ledger/payment history

Delivery coordination	Informal route sharing	Route and stop records
Tiffin identification	Physical/manual identification	QR-linked box status
User support	Repeated questions to owner	Rule-based assistant

The system therefore improves organization and traceability even where the underlying business process remains partly manual. For example, the payment itself is still made through UPI and verified by the mess owner; MESSIFY digitizes the submission and approval workflow rather than claiming to replace the payment network.

Similarly, delivery tracking is implemented as a lightweight route and stop state system. It provides a basis for showing delivery progress, but it should not be presented as a full navigation or vehiclerouting solution. These boundaries make the thesis technically accurate and aligned with the actual project implementation.

5.1 TEST RESULTS

The testing results indicate that the main functional workflows can be connected through the Firebasebacked data layer. Authentication provides the entry point, role-based pages expose the relevant functionality, and Firestore records provide shared state between student, mess-owner and delivery modules.

Table 5.2 Test Results and System Evaluation

Area	**Observed Result**	**Evaluation**
Authentication	Login and registration flow integrated with Firebase Authentication.	Functional
Mess discovery	Mess profiles can be loaded and displayed for browsing.	Functional
Subscriptions	Plans and subscription states are stored in Firestore.	Functional
Payments	UPI proof submission and owner approval/rejection are represented.	Functional
Meal management	Daily menu and skip records are supported.	Functional
Billing	Ledger records and payment history can be retrieved.	Functional
Delivery	Routes, stops and progress states are supported.	Functional

QR	Tiffin identifiers can be scanned and stored.	Functional
Notifications	System events can generate notification records.	Functional
AI assistance	Rule-based suggestions and support responses operate within defined rules.	Functional / Limited

The most important result is the integration of recurring-service operations. A student action such as submitting payment is not isolated: it creates a record that the mess owner can review. Likewise, a meal skip is stored against a subscription and date, making it visible to the service side. This shared data approach is a key improvement over phone-based or notebook-based coordination.

The research paper reports that the system accomplishes user authentication, mess finding, subscription, meal/menu tracking, skipping, invoicing, payments, QR identification, delivery, notifications and AI assistance. It also notes that the AI implementation is currently simplistic and operates within predefined parameters.

5.2 FUNCTIONAL OUTCOME ANALYSIS

From the student perspective, the major outcome is convenience and visibility. The student can access menu information, subscription status, payment-related information, meal skipping and deliveryrelated status through a common interface. From the provider perspective, the result is operational organization: menu updates, subscriber information, payment approvals, meal skips and delivery records are presented through dedicated modules. The delivery partner receives an assigned route and QR scanning capability, while the administrator receives broader system visibility.

The AI-assisted outcome should be evaluated according to its implemented scope. The assistant can respond to predefined service-related intents, while meal suggestions use current contextual information. This provides useful application-level assistance, but it does not establish predictive performance. The result is therefore best described as a working intelligent-assistance layer that can later be replaced or extended with trained recommendation and prediction models.

Student experience: The student receives a single dashboard for active subscriptions and current meal information. The skip button makes meal flexibility explicit. Recommendations and AI suggestions provide additional guidance without interrupting the core service workflow.

Mess-owner experience: The owner dashboard converts several recurring administrative tasks into visible modules. Active subscribers, earnings and meal skips can be monitored. Payment approvals provide a clear checkpoint before subscription activation. Menu management creates a single source of current meal information.

Delivery experience: Route records provide an ordered list of stops, while QR scanning connects a physical tiffin box to a digital identity. The current-stop index creates a simple progress indicator that can be observed by the system.

Administrative experience: The administrator can inspect high-level records and provide oversight.

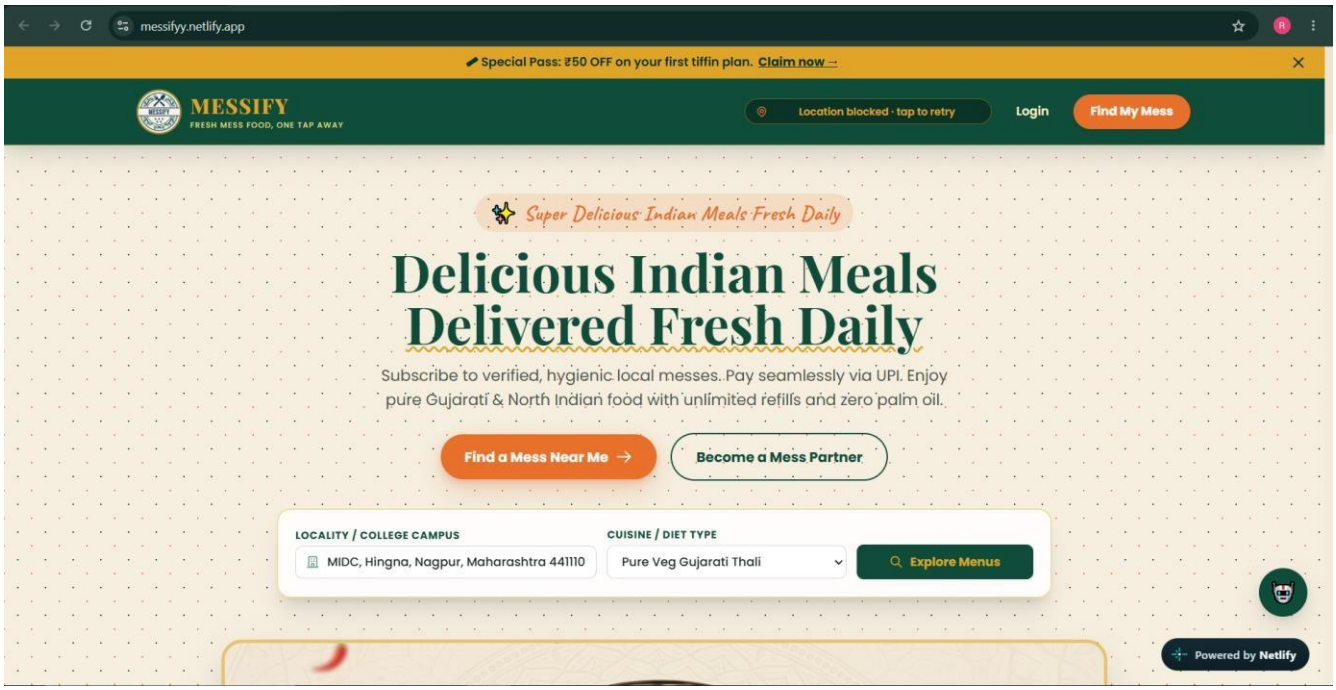
This role is important when the number of providers and customers increases.

AI-assisted experience: The current intelligent features focus on convenience rather than prediction. The assistant answers common questions using predefined rules, while meal suggestions use current context and available menu information. This design is transparent and deterministic, making it easier to test than an opaque model without training data.

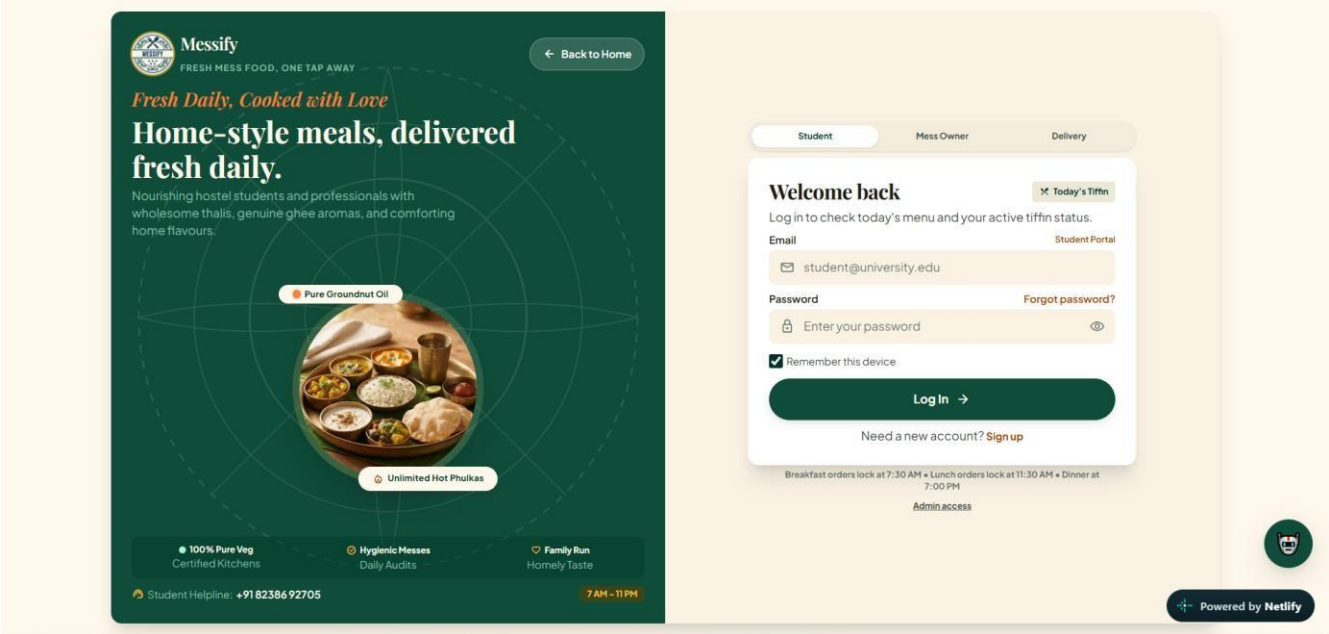
Overall, the project demonstrates how a recurring service can be decomposed into connected digital modules. The strongest contribution is not a single feature, but the integration of multiple operational steps into a consistent system.

5.3 SCREENSHOT INSERTION PAGES

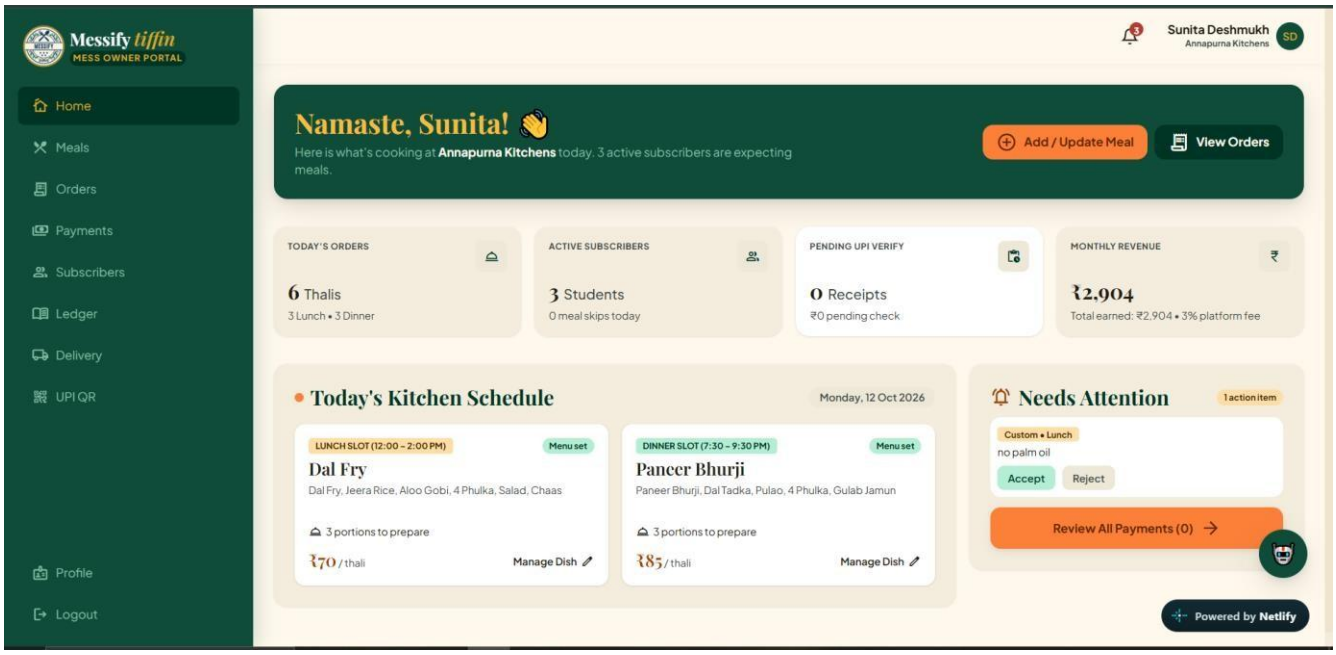
The following screenshot slots are intentionally reserved for actual screenshots from the final running project. Screenshots should be captured from the final build and inserted with figure numbers



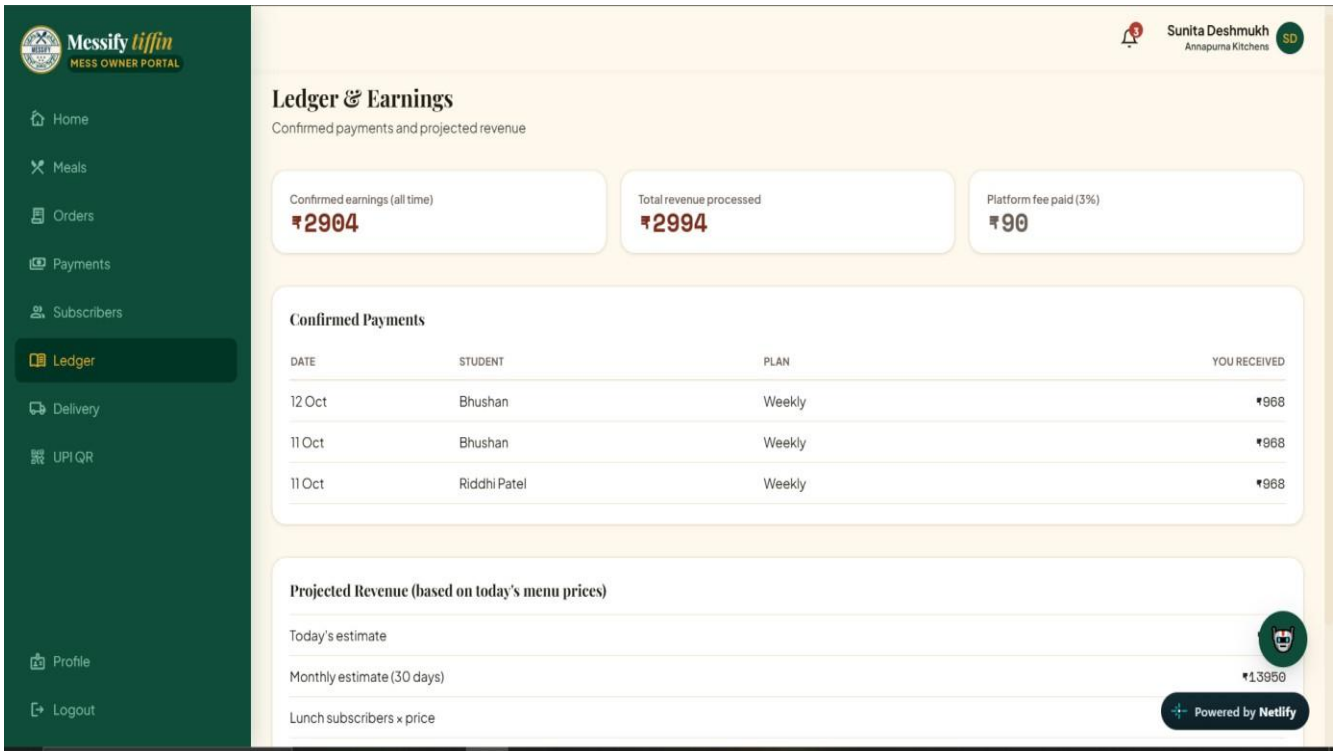
Photograph 5.1 - Mess Portal Section



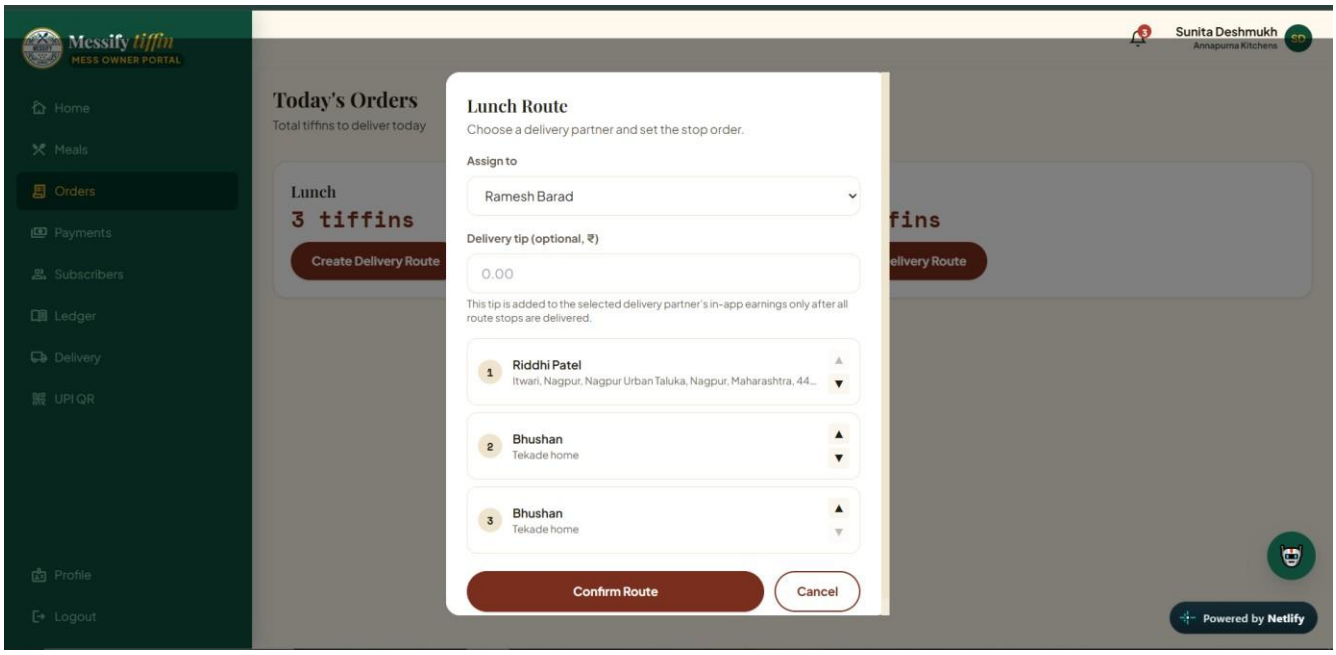
Photograph 5.2 User Login section



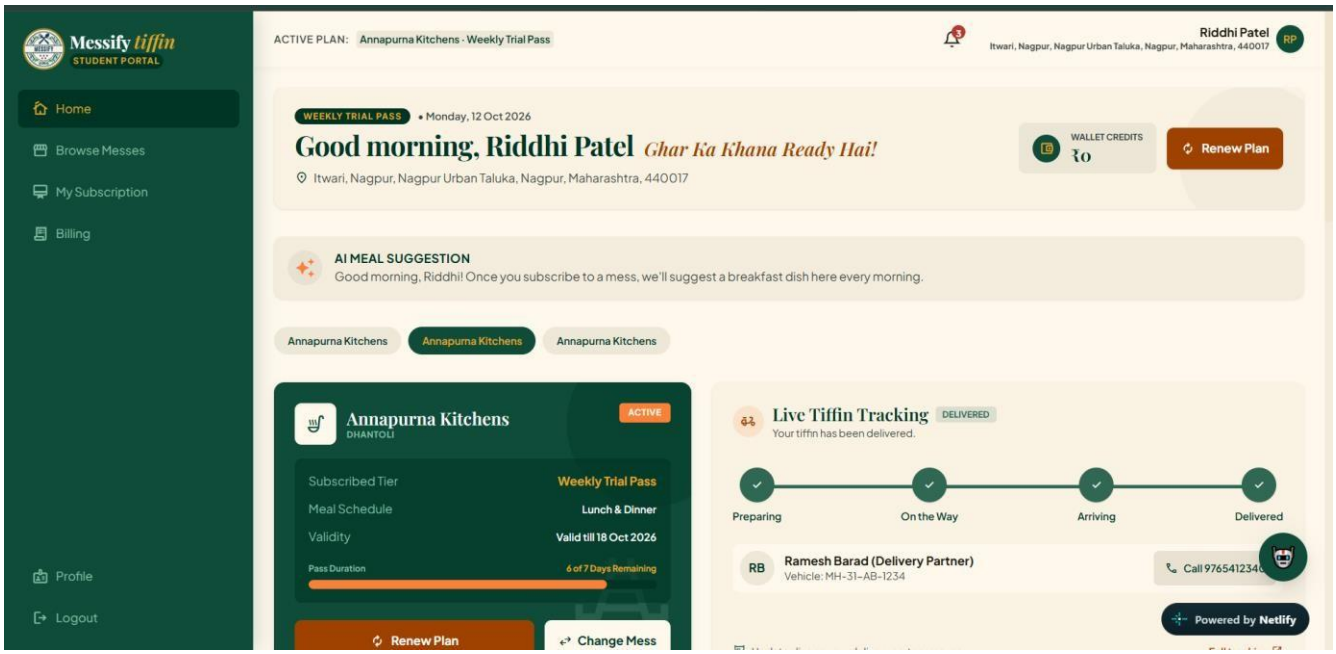
Photograph 5.3 Mess owner Dashboard



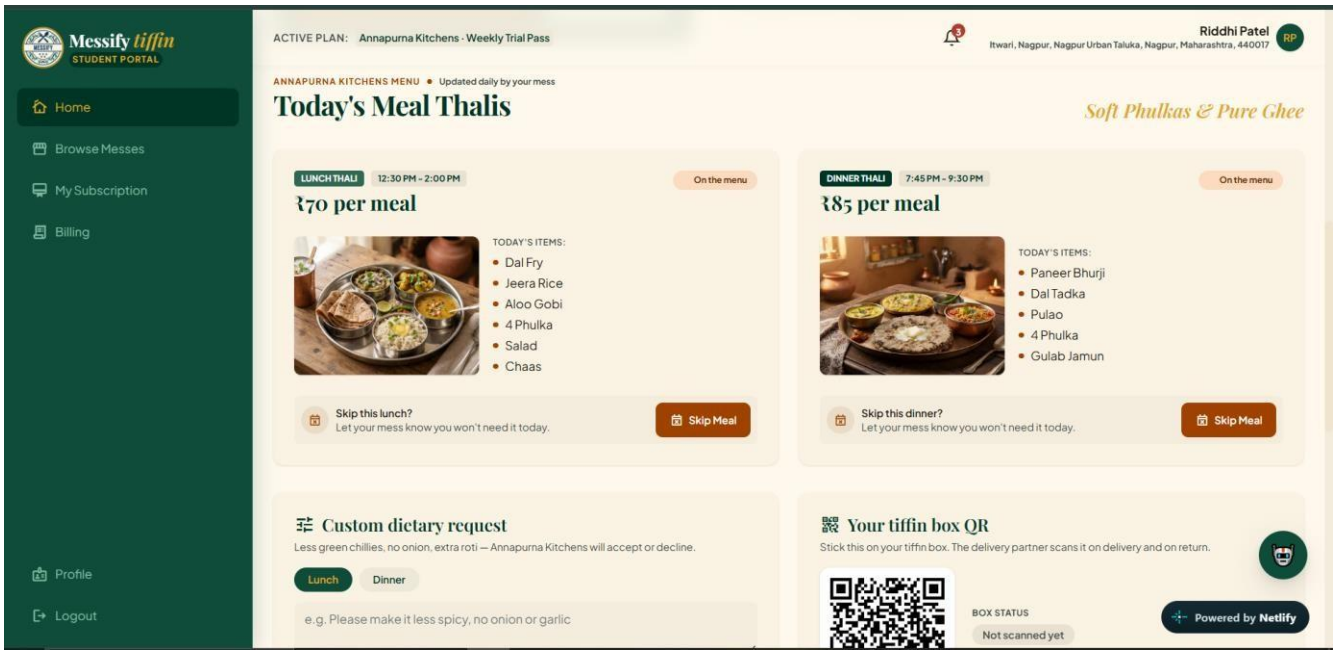
Photograph 5.4 Mess Owner Ledger & Earning Data



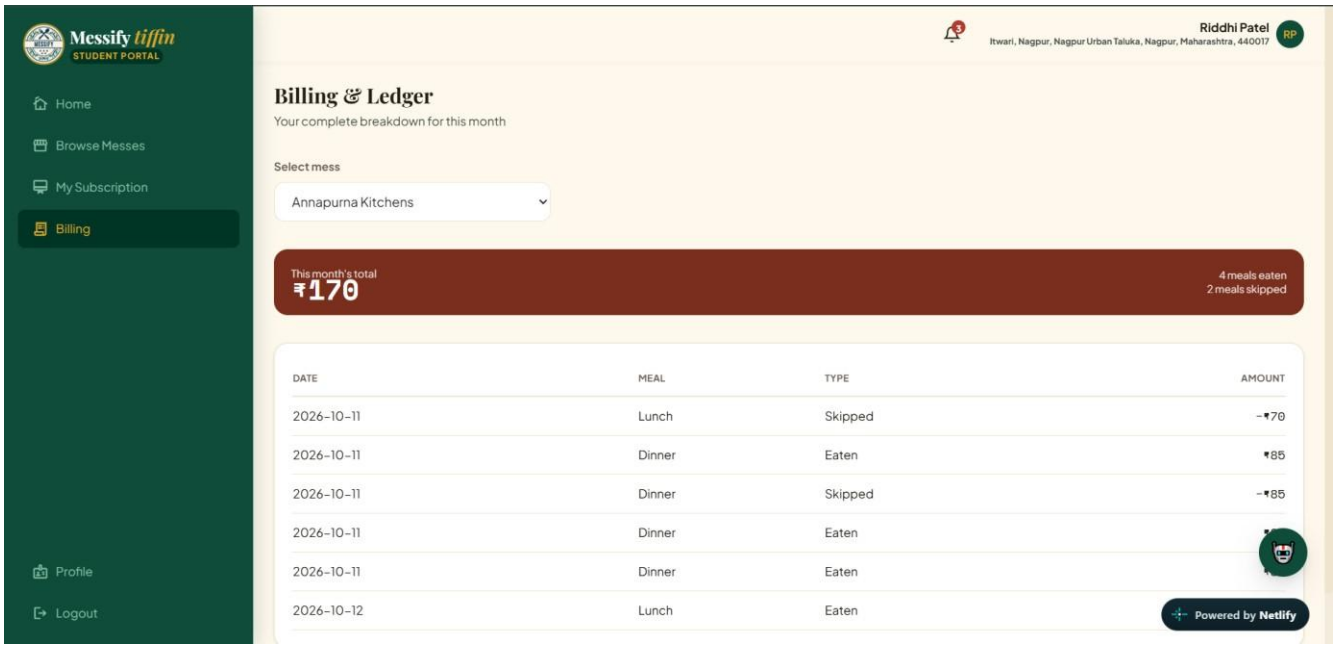
Photograph 5.5 Route assign for Mess Owner



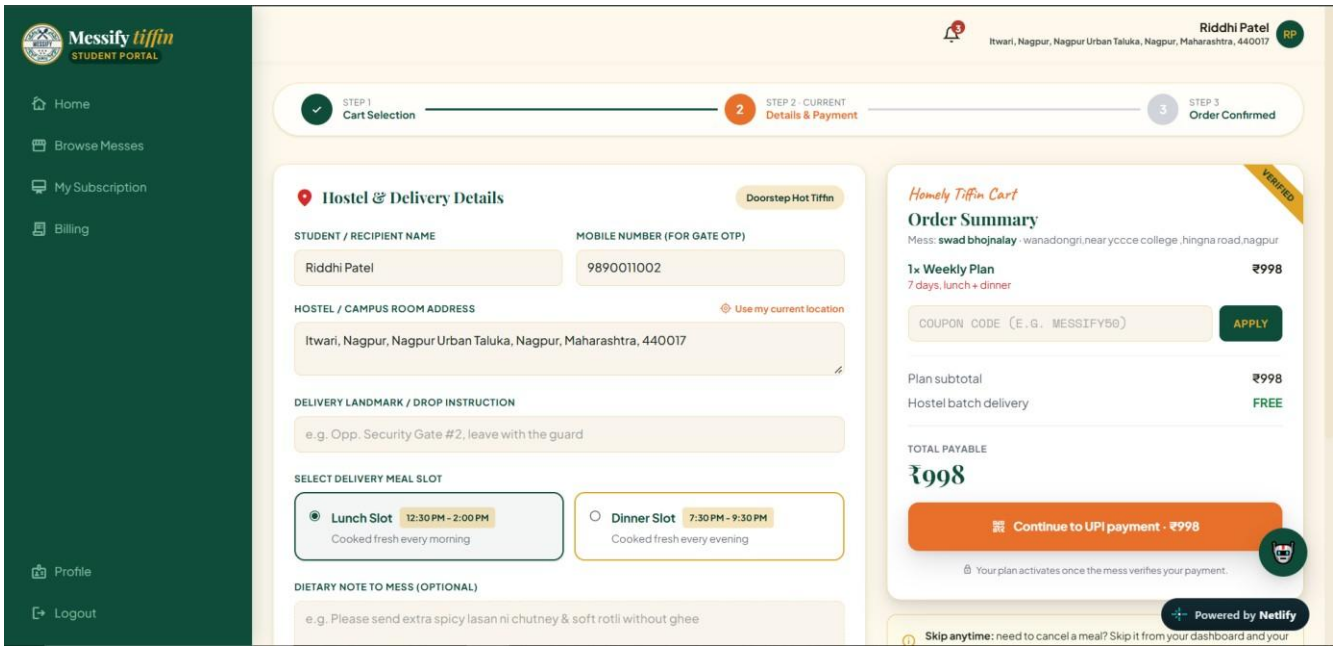
Photograph 5.6 Student Dashboard



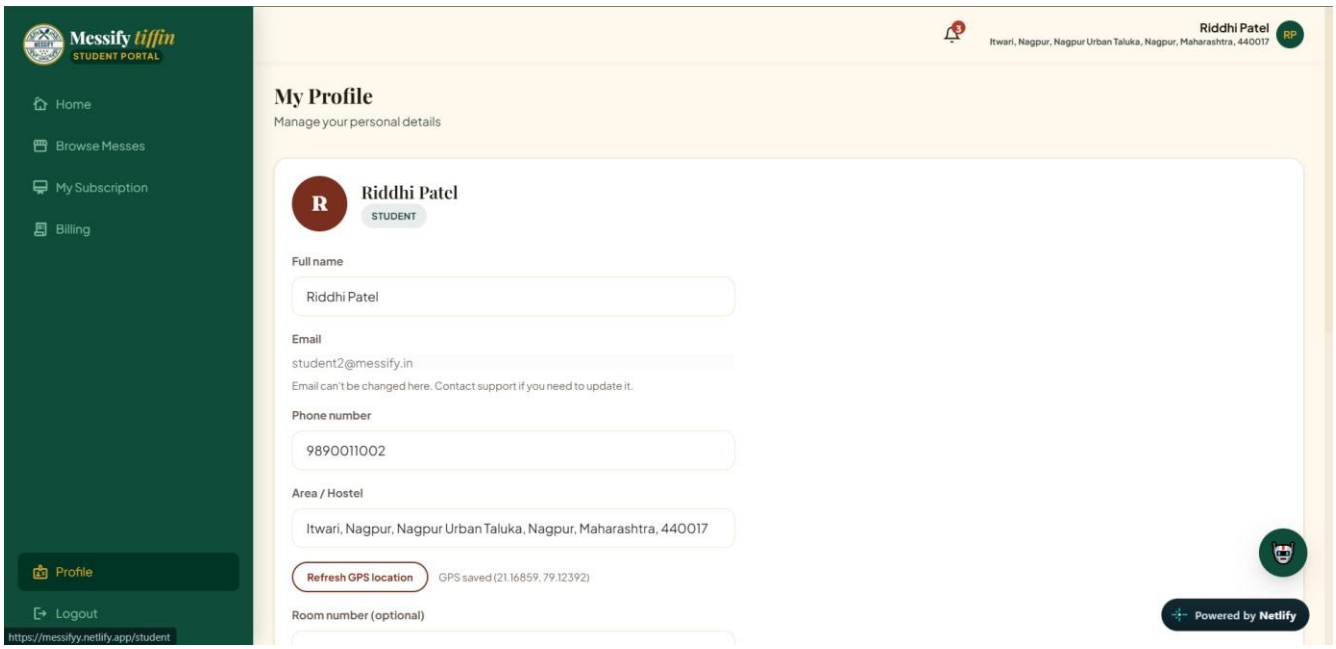
Photograph 5.7 Meal Update and Custom Dietary Request interface



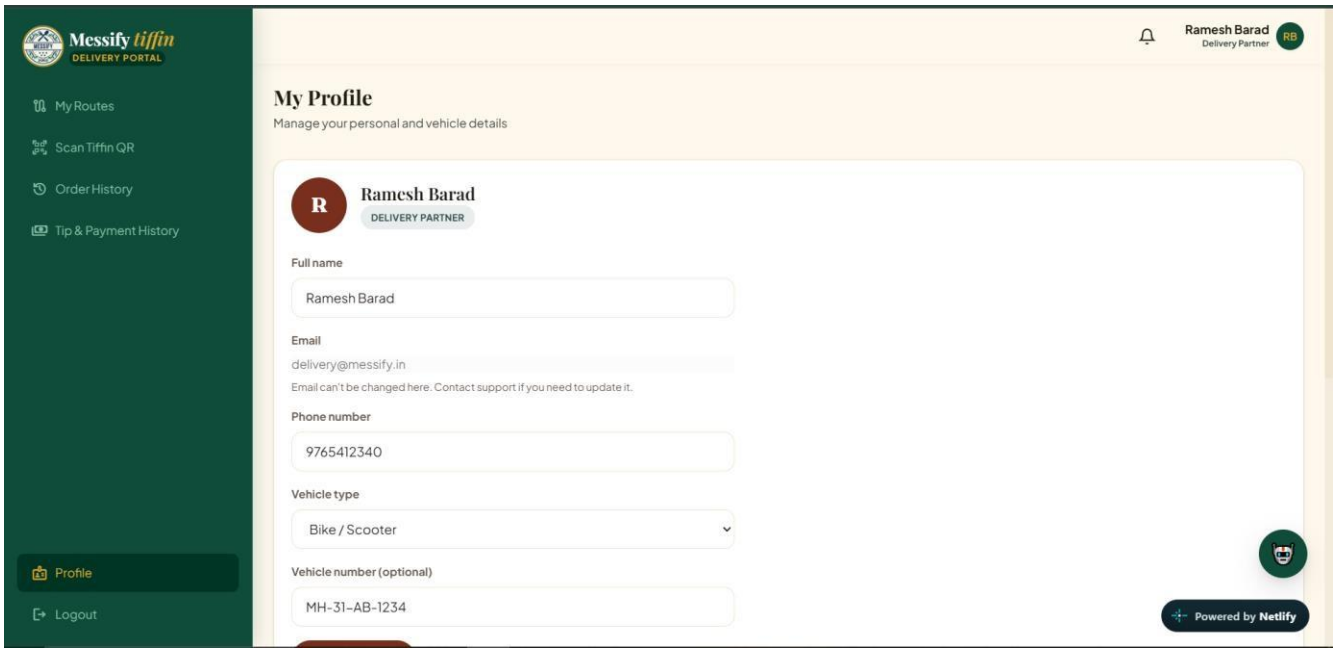
Photograph 5.8 Billing and Ledger for Student.



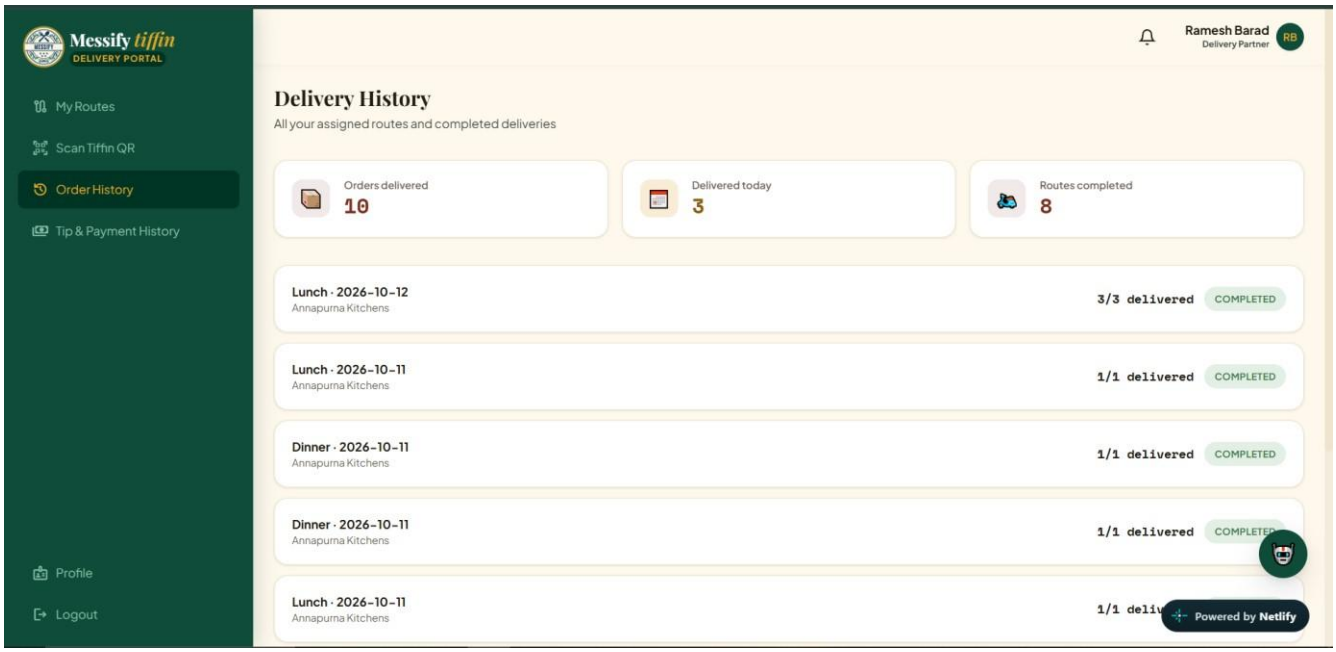
Photograph 5.9 Payment Section



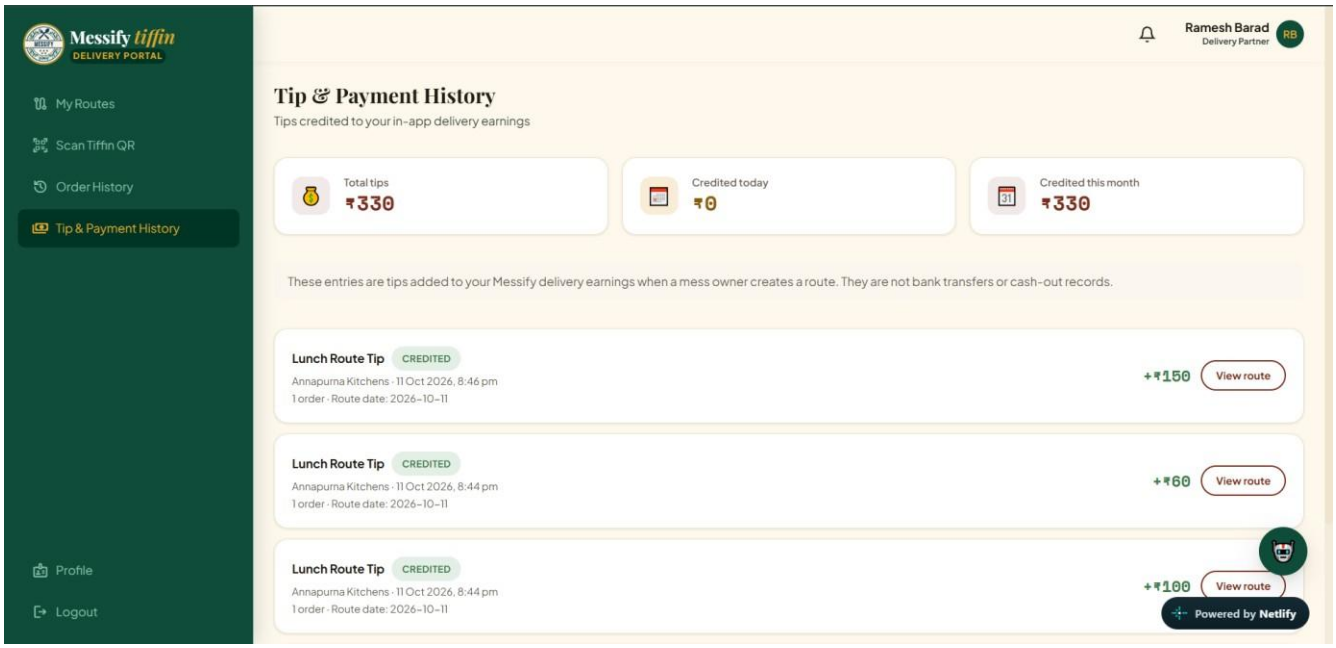
Photograph 5.10 Profile Section of Student



Photograph 5.11 Profile Section of Delivery Partner



Photograph 5.12 Delivery History



Photograph 5.13 Payment History of Delivery Partner

6. CONCLUSION

The project demonstrates that a recurring mess and tiffin service can be represented as a set of connected digital workflows rather than as a collection of independent manual tasks. Authentication establishes identity, role-based interfaces separate responsibilities, Firebase provides shared cloudbacked records, and operational modules connect subscription, meal, payment and delivery activities. The resulting system provides a practical academic prototype for a domain that is often managed through informal communication. The project also establishes a clear research direction. The present rule-based AI layer provides contextual assistance and simple personalization, while the data structures created by the application can support future machine-learning work. By distinguishing implemented functionality from future intelligence, MESSIFY presents a realistic and technically defensible foundation for further development.

MESSIFY presents a centralized digital solution for managing the recurring activities of college mess and local tiffin services. The project addresses a practical gap between traditional manual coordination and modern digital food-service platforms by focusing on recurring subscriptions, daily menus, skipped meals, billing, delivery and reusable tiffin management.

The implemented web application uses React.js for the front end and Firebase services for authentication and cloud data management. Its role-based design connects students, mess owners, delivery partners and administrators. The project also incorporates QR-based tiffin identification, delivery route management, notifications and a lightweight AI-assisted experience.

A key conclusion is that the AI aspect should be understood accurately. The current project demonstrates rule-based assistance and contextual meal suggestions rather than a trained machinelearning system. This provides a useful foundation because the application already collects or represents the types of data that could later support prediction: subscriptions, skips, menus, payments and delivery records.

The project therefore achieves its primary academic objective: it demonstrates how a real-world messmanagement problem can be analyzed, decomposed into functional modules, implemented using modern web and cloud technologies, and enhanced with transparent AI-assisted features. With future data collection and model development, MESSIFY can evolve toward demand prediction, food-waste reduction, stronger personalization and optimized delivery.

In conclusion, the system establishes a practical foundation for a smarter mess-management ecosystem while clearly identifying the technical and research work required for a production-scale intelligent platform

6.1 FUTURE SCOPE

The future development of MESSIFY can proceed in stages. The first major stage is the collection and preparation of historical data. Once sufficient records of subscriptions, menu selections, skips and meal demand are available, a machine-learning model can be evaluated for demand prediction. The model could help estimate expected meal counts for individual days and reduce the difference between prepared meals and actual consumption.

A second stage can focus on personalization and optimization. Recommendation models could consider previous selections, ratings, skips and dietary preferences. Delivery routes could be optimized using location, delivery windows and route constraints. Payment processing could be integrated with a secure gateway and automated reconciliation. The AI assistant could also be connected to authenticated application data through a controlled retrieval mechanism so that it can provide richer support without exposing private records.

Other future extensions include automated monthly invoices, reusable tiffin-return tracking, provider analytics, food-cost analysis, multilingual interfaces, richer feedback mechanisms and stronger administrative reporting. These enhancements are consistent with the research direction identified in the supplied paper, particularly demand prediction, personalized recommendations, food-waste prediction and delivery optimization.

The next stage of MESSIFY can introduce genuine machine-learning capabilities after sufficient historical data becomes available. Meal demand prediction can use active subscriptions, historical skips, day of week, menu type, holidays and previous consumption to estimate the number of meals likely to be required. This could help mess owners prepare food more accurately and reduce waste.

Personalized meal recommendation can be extended beyond rating and area. A model could learn from meal selections, skips, ratings, dietary preferences and historical orders. Nutritional information could also be incorporated, provided the data is reliable and the system avoids making unsupported health claims.

Delivery optimization is another major direction. The current route model can be extended into a routing engine that considers geographic distance, delivery windows, vehicle capacity and changing stop status. Real-time GPS could provide more accurate estimated arrival information.

Payment processing can be upgraded from manual UPI proof verification to a secure payment gateway with webhooks and automated reconciliation. Such an integration would require stronger backend security and compliance considerations.

The AI assistant can be replaced or augmented with a retrieval-based or large-language-model system backed by authenticated application data. A production design should ensure that the assistant cannot expose private student information or perform unauthorized actions.

Additional future modules may include attendance-based meal prediction, automated monthly invoices, QR-based reusable-container return tracking, provider analytics, food-cost analytics, feedback and ratings, multilingual assistance, and institutional mess management for universities.

The supplied paper similarly identifies demand prediction, personalized meal recommendations, foodwaste prediction and delivery optimization as future development directions.

REFERENCES

[1] Y. Fu, L. Cai, R. Wu, Y. Kang & Y. Zhao, “Nutrient-Aware Personalized Meal Recommendation Using Structured Food Knowledge and Constraint Verification,” 2026.

[2] R. Singh & P. Dwivedi, “A Health-Aware Food Recommender System Using an Interestingness Score and Random Forest,” 2026.

[3] S. Chaudhari, A. Rane & A. R. Kumar, “Personalizing Nutrition and Recipe Recommendation Using Attention Mechanism with an Ensemble Model,” 2025.

[4] A. M. Saad, M. R. H. Rahi, M. M. Islam & G. Rabbani, “Diet Engine: A Real-Time Food Nutrition Assistant System for Personalized Dietary Guidance,” 2025.

[5] A. M. Saad et al., “Intelligent Diet Recommendation System Powered by Artificial Intelligence for Personalized Nutritional Solutions,” 2025.

[6] D. Tsolakidis et al., “An AI-driven Multivariate Approach for Personalized Healthy Eating Recommendations,” 2026.

[7] MESSIFY Project Source Code, React.js / Firebase implementation, supplied as the project ZIP for this thesis.

[8] MESSIFY: An AI-Powered Smart Mess Management System, research paper supplied by the project team, 2026.

Reference note: The first six bibliographic entries above are reproduced from the supplied MESSIFY research paper. No DOI, journal volume, issue, page range or publisher information has been added where it was not present in the supplied material. This avoids inventing bibliographic details.

RESEARCH PAPER PUBLICATIONS

The research paper based on this work, “Messify- An AI Powered Mess Management System”, has been prepared for publication.

[the front page of the published paper here once it is published or accepted.]

CERTIFICATE OF RESEARCH PUBLICATION

[Insert the certificate of research publication here.]